

Waste identification and elimination in information technology organizations

Osama Al-Baik · James Miller

Published online: 5 February 2014

© Springer Science+Business Media New York 2014

Abstract In this paper, we propose a new model to classify wastes in IT organizations. In the beginning, we discuss the potential reasons behind the relatively low success rate of lean initiatives in knowledge-based industries in general and in IT organizations in particular. The virtual nature of business processes in IT organizations calls into question the applicability of Toyota’s categorization of physical wastes in IT settings. Then, through a real-life project, we develop a new model of waste categorization for the operation of a “medium-sized” IT department. In addition to the new classifications specific to IT, we discuss suitable elimination strategies and how they have improved the daily operations of the organization by reducing the lead-time by 56–60 %, increasing customer satisfactions by 15.7 %, and saving hundreds of thousands of the operational cost. Finally, we emphasize the need to think lean when developing the waste elimination strategies by eliminating the root cause of the waste and not the subsequent wastes.

Keywords Lean thinking · Lean production · Lean software development · Waste in IT · Software process improvement · Agile

1 Introduction

Waste identification and elimination are important and integral principles to Lean production, which was defined by Womack (2007) as:

Lean manufacturing, lean enterprise, or lean production, often simply, Lean, is a production practice that considers the expenditure of resources for any goal other than the creation of value for the end customer to be wasteful, and thus a target for elimination.

Waste elimination steps have been successfully used to increase the productivity of manufacturing organizations. These steps help to reduce time-to-market and the cost of

Communicated by: Nachiappan Nagappan

O. Al-Baik · J. Miller (✉)
Department of Electrical & Computer Engineering, University of Alberta, Edmonton, Alberta, Canada
e-mail: jimm@ualberta.ca

O. Al-Baik
e-mail: albaik@ualberta.ca

delivering a product to customers. Primary evidence of this has been reported by Toyota, Japan, which uses its Toyota Production System (TPS) to identify and eliminate manufacturing waste (Liker and Hoseus 2008; Magee 2008). Having gone through many decades of development and enhancement, the Lean philosophy has been proven successful in the manufacturing industry (Magee 2008); however, the feasibility of its application in other different industries has not been completely established (Staats and Upton 2011; Spearman 2004).

This paper proposes a new model for categorizing wasteful processes and activities in IT organizations. The new model explains the types of wastes, where they exist, and how to eliminate them. Based upon the results of research studies, produced by Shelton (2010), DeMarco (2011), Wang et al. (2012), and Schwaber & Sutherland (2012), a new model has been developed around the reasons for high costs in certain phases of IT projects lifecycles, IT operations, and the reasons behind IT project failures.

A study of the DuPont organization showed that 75 % of commercial software features were not needed (Shelton 2010). In addition, during a speech, Jim Johnson, CEO of the Standish Group, stated that the results of the DuPont study showed that “On average, 45 % of software features are never used, and only 20 % of features are used always or often” (Shelton 2010, p. 4).

This paper seeks to demonstrate the validity and value of the proposed model to identify and eliminate wasteful processes by applying it to the operations of a “medium-sized” IT department¹. The model has proven successful and helped ORGUS in reducing operational costs and service delivery lead-time by 56 to 60 %; on the other hand, ORGUS realised a higher customer satisfaction rate by 15.7 %. During the lean initiative in ORGUS, the investigators identified 20 duplicate services provided by different teams, more than one hundred services provided by inappropriate teams, and about 15 services that should not be provided at all.

The main contributions of this paper can be summarized as follows:

- The model has been empirically evolved based upon IT work in real life-project,
- The model has considered both, software development and IT service operations.
- To the best of our knowledge, none of the scholars in the existing literature has provided a description of “how to” identify and eliminate wasteful processes and activities. This report provides a detailed narrative of waste identification and elimination strategies that have proven successful in ORGUS.
- The report helps IT organizations in their decisions when implementing a Lean initiative in general, and waste identification and elimination in particular, as well as illustrating the challenges that come with a decision to implement it.

The lean initiative in ORGUS was tackled as a standalone project that lasted for 18 months, during which the project team was composed of two groups: 1) a core project team of eight members, and 2) a supporting group, where other resources were engaged when necessary. The total number of employees involved in this project was 93 members, who worked at various periods of times ranging from 1 day to 6 months. The investigators were involved in around 100 meetings that varied from 1 h to 3 h. The study included review and analysis of a

¹ The identity of the organization under study is kept anonymous and is explicitly omitted from this paper. This is an agreement by the organization and the research group. Hence, for the remainder of the paper, we will refer to the company by an arbitrary name ORGUS.

total of 130 projects carried out in ORGUS, composed of 70 small projects, 35 medium project, 15 large project, and 10 extra large projects.

The remainder of this paper is structured as follows: Section 2 provides a summarized benchmarking and literature review about the seven types of waste, the core of Toyota Production System (TPS) and the previous literature review about types of waste in IT. Section 3 provides a detailed experience report in which we proposed new classifications for wastes in IT organizations; the new model has been implemented in a “medium-sized” IT department. Section 4 provides statistical information about the performance improvements achieved in the company, and finally in Section 5, conclusions and recommendations are drawn from both the literature reviews as well as from the organization we studied.

2 Benchmarking Led to Literature Review

Applying and implementing Lean philosophy is not an overnight process. In order to have a successful Lean journey, enough time must first be allocated to analyze and understand the nature of the organization, its business processes, and the environment in which the organization operates. A significant amount of research about applying Lean has shown credible results (Staats et al. 2011; Poppendieck and Poppendieck 2003). However, these scholars have not described or explained how the wasteful processes have been identified and which waste elimination strategies have been used or were useful, evidences can be found in (Wang et al. 2012). Several researchers (Staats and Upton 2011; Spearman 2004; Shah and Ward 2007) acknowledge that waste identification is not an easy task, and others point to a lack of “how-to” methods for identifying and eliminating waste without further explanation.

As part of the benchmarking, and in an attempt to provide cheaper, faster and higher quality IT services, we examined recently published empirical studies and experience reports that included narrative of the initiatives to eliminate wastes in IT organizations; see (Wang et al. 2012) for the actual reports. We have noticed that:

- 1) From the 30 experience reports, Wang et al. (2012) claimed that eight companies implemented the “Eliminate Waste” principle, and five of the eight organizations operate in IT-related business domains. After reviewing the actual reports, we discovered that one of studies reported the development of a software application to eliminate wastes in a restaurant (Rand and Eckfeldt 2004), another report did not try to implement “Eliminate Waste” at all (Nottonson 2011), and the remaining three reports (de-Ste-Croix and Easton 2008; Yap 2006; Packlick 2007) used Poppendick’s classification of wastes and simply stated that wastes were discovered and improvement opportunities were identified.

Hence, we could not find any existing studies to help us in tackling the waste identification and elimination initiative, which supports our claim in the third bullet point of the identified contributions list.

- 2) The waste classifications specified by Poppendieck and Poppendieck (2003, 2006) were mapped from Ohno’s waste categorization. To the best of our knowledge, no waste categorizations or classifications have evolved purely based upon IT work, except two types of wastes: “Unnecessary Work” and “Avoidable Rework”, which were identified by Petersen and Wohlin (2009) as part of interview questions.

In comparison to manufacturing, IT processes and activities do not produce physical wastes; (Staats et al. 2011), thus, the wastes in IT should have been identified and categorized based upon IT work rather than being mapped from manufacturing. This study identified nine different categories of wastes based upon a real-life project in IT

organization, which supports our claim in the first bullet point of the reported contributions list.

- 3) Petersen and Wohlin (2010), and Poppendieck and Poppendieck (2003) emphasized that significant changes to the underlying culture of the organization and its way of doing work is essential to implement lean successfully. In addition, Poppendieck and Poppendieck (2003) reported that software development organizations have not been successful in implementing lean as the required changes to the organizations' culture and habits are not easy to implement.

Successful implementation of lean in general and waste identification in particular was challenging. This study reports the implementation of waste identification and elimination to both, software development and IT services through the use of ethnographic methodologies, where the authors were involved in a real life project for 18 months, which supports our claim in the second bullet point of the reported contributions list.

- 4) Poppendieck and Poppendieck (2003) on the other hand, emphasized those organizations that understood "Lean thinking", were able to develop context-dependent practices that facilitate the achievement of "sustainable performance improvement". They believe that when imitating proven techniques from other disciplines to software development, the problems most likely arose because of imitating the practices (context-specific) rather than the principles.
- 5) Shah and Ward (2007) calls into question the level of understanding possessed by different scholars in regard to "Lean thinking". Hence, we started reviewing the original Lean essence from the manufacturing perspective to have in-depth understanding of Lean principles. Waste elimination is one of the Lean principles; however, waste classification and identification are practices to implement the principle in specific settings. That is, context-dependent should differ from one environment to another (Poppendieck and Poppendieck 2003; Wang et al. 2012).

This study shed some lights on how the practices were determined based upon several factors in ORGUS, including but not limited to: culture, resources, budget constraints, and time constraints, which supports our claim in the fourth bullet point of the reported contributions list.

2.1 Types of Waste

The main focus of this paper is the identification and elimination of wastes in both, software development and IT service operations in a medium size IT department. In order to provide a comprehensive overview of wastes and to emphasize the differences between IT and manufacturing processes, this section is decomposed further into two sections, section 2.1.1 provides a description of wastes in manufacturing, and section 2.1.2 discusses the wastes in IT organizations and the differences between the nature of IT wastes and manufacturing wastes.

Wastes are defined as unnecessary activities that add cost or time without adding value (Tapping 2002; Dennis 2007). The goal of most organizations is to provide the highest quality product, in the shortest time, and at the lowest price. Customers, however, require additional attributes beyond product or service quality, price, and time to delivery; therefore, organizations implementing Lean need to focus on a

broader set of customer expectations. For example, many Lean organizations have added safety, environment, and morale to their goals (Dennis 2007).

2.1.1 Wastes in Manufacturing

The founder of Toyota Production System (TPS), Taiichi Ohno identified seven general types of waste in manufacturing (Womack 2007), where products vary significantly from industry to industry. The wastes that are found in manufacturing environments are, however, quite similar. The main reason for this similarity lies in the nature of the physical and tangible wastes. The seven most common types of waste found in manufacturing environments are:

1. **Overproduction.** Overproduction occurs when producing an item that is not intended for immediate use or sale is manufactured (Tapping 2002). Overproduction is the source of many other types of waste. In fact, Ohno declared overproduction to be the root cause of other kinds of wastefulness and manufacturing evil (as cited in Dennis 2007).
2. **Waiting.** Waiting waste occurs when a worker has to wait for the work to be released from another worker, another manufacturing process, or for material to be delivered. Goldratt has explained that 1 h lost in a bottleneck process is 1 h lost to the entire manufacturing output and one which can never be recovered (Goldratt and Cox 1992).
3. **Transportation.** A transportation waste results when material is moved more than is necessary; this type of waste is usually caused by an inefficient and poor workplace layout. Although transportation is a non-value adding activity, it is a necessary process, however, organizations should strive to minimize material movement.
4. **Over-processing.** Over-processing waste relates to spending more time, efforts, and resources to produce higher quality product than is required by the customer, or producing these products using inappropriate tools. Many organizations use high precision equipment where simpler tools would be sufficient (Tapping 2002). Over-processing is considered the most difficult type of waste to identify and eliminate.
5. **Inventory.** Inventory waste is defined as the retention of unnecessary raw materials, “Work in Progress” (WIP), and finished products. WIP is a direct result of overproduction and waiting. Unnecessary inventory tends to hide problems that must be identified and resolved in order to improve an organization’s performance. Unnecessary inventory increases lead times, consumes otherwise productive floor space, delays the identification of problems, and inhibits communication (Dennis 2007).
6. **Excess Motion.** Excess motion waste is any human “motion” that is unnecessary to the successful completion of an operation. This waste is related to workplace ergonomics (Dennis 2007) and includes all instances of bending, stretching, walking, lifting, twisting, and searching for parts or tools. Jobs that need excessive human movement should be analyzed and redesigned to reduce the required amount of motion.
7. **Defect or Rework Waste.** Defective products need to be fixed or reworked, this waste category also includes productivity losses and waiting waste associated with disrupting the production flow to deal with the defects or rework. A large body of literature has been developed on the cost of quality, and specifically the cost of repairing defects and scrap (Besterfield 2009).

Lean Thinking theorists and Lean Manufacturing practitioners have recently added under-utilization of employees as an eighth type of waste (Dennis 2007). Organizations should invest in utilizing employees' creativity so that they can work towards eliminating the other seven wastes and continuously improve their performance.

2.1.2 Wastes in IT Organizations

IT organizations have not experienced the same level of success with Lean implementations as has been achieved by organizations in the automobile industry. The lack of strategies that have been used to identify and eliminate waste is one of the important factors responsible for this disappointing performance.

IT organizations continue to fail to deliver reliable systems within scope and budget, and on schedule (DeMarco 2011), thus the software industry is still considered to be a relatively immature industry. The success stories of Lean Manufacturing in the automobile industry have prompted IT organizations to begin learning about, and implementing, Lean principles and strategies.

As Toyota and other world-class organizations have come to realize, customers will pay for value-added work, but never for waste (Hino 2002). To successfully eliminate waste, it is important to understand what the wastes are, where they exist, and how to attack them. The key to success in IT Lean initiatives is to understand the differences between IT and manufacturing environments: the Lean initiative is not a “cut and paste” project: rather, it must be developed based upon the nature of a specific business.

As reported by Wang et al. (2012), there were various attempts to apply Lean in IT, such as de-Ste-Croix and Easton (2008), Yap (2006), and Packlick (2007), these attempts, however, have been dependent on identifying waste based upon the proposed models by Poppendieck and Poppendieck (2003, 2006) that were mapped from manufacturing rather than being evolved from IT work. The mapped types of waste found in software development and in IT operations are illustrated below in Table 1.

Table 1 Mapping of wastes to IT operations and software development

The seven wastes of manufacturing	The eight wastes of IT operations	The seven wastes of software development	
Ohno (1988)	Waterhouse (2008)	Poppendieck and Poppendieck (2003)	Poppendieck and Poppendieck (2006)
Inventory	Inventory (excess)	Partially done work	Partially done work
Extra processing	Non-value adding processing	Extra processes	Relearning
Overproduction	Overproduction (over provisioning)	Extra features	Extra features
Transportation	Transportation	Task switching	Handoffs
Waiting	Waiting	Waiting	Delays
Motion	Motion (excess)	Motion	Task switching
Defects	Defects	Defects	Defects
	Employee knowledge (unused)		

From Table 1, it is obvious that the wastes in IT operations by Waterhouse (2008) and the wastes in software development by Poppendieck and Poppendieck (2003, 2006) have been mapped from the waste categorization that was established purely for manufacturing, specifically, the automobile industry.

Poppendieck and Poppendieck have revised their mapping only after 3 years of publishing their original waste categorizations, where the *Extra Processes*, *Task Switching*, and *Waiting* have been replaced, respectively by *Relearning*, *Handoffs*, and *Delays*. It is noticeable that *Task Switching* has not been completely disappeared from the new mapping as it has replaced the *Motion* waste. This revisiting of the waste classifications is a clear indication of the need to develop more stable and suitable model for IT organization that evolves based upon IT core work.

Almost every attempt in IT has focused on the administrative aspects of the work done in IT, ignoring the actual core work that is done within IT core processes, this can be concluded from recent articles (Staats et al. 2011; Wang et al. 2012). For example, Staats et al. (2011) identified the location of a printer in a different floor as a waste in an IT organization. Despite the fact that the location of the printer is an obvious *motion* waste, however, it is not an IT core waste; rather, it is an administrative waste. In order to understand what the wastes are, where they exist, and how to attack them, we have developed a new model to classify IT-specific waste.

3 Experience Report: Waste Identification and Elimination in ORGUS

The aim of this research study is to answer the following questions:

- Are the waste categories identified by Toyota appropriate for IT service organizations?
- Can we develop a better understanding of the types of waste produced by IT service organizations?
- How can we identify wasteful processes in IT service organizations? And,
- How can we eliminate these wasteful processes?

The remainder of this section is structured as follows: section 3.1 provides a description of the study context. Section 3.2 provides an overview of the action research methodology used in this project and provides description of the main methodology phases. Section 3.3 outlines the project approach that has been used during the implementation of the waste identification and elimination initiative, it also describes the techniques used to develop and visualize high-level process flows. Section 3.4 describes how the model was evolved and provides a detailed description of the identified nine categories of wastes, in addition to examples of wastes that were identified under each category along with their elimination strategies.

3.1 Study Context

ORGUS is an internal “medium-sized” IT department that provides services for 35,000+ end users, has a staff exceeding 250, and an estimated annual budget of approximation \$30 million. ORGUS provides a wide set of IT services divided and categorized into 24 groups, including, but not limited to: Application and website hosting, workgroup applications, system administrations, networking, project management, IT service management,

desk-side support, physical infrastructure and datacenters, and in house application development. The services in ORGUS are divided into two main categories based upon ITIL framework (Addy 2007):

- 1) *Request for Change*: this type of service requires a change that usually affects more than one individual and takes long time to implement, typically weeks or even months, as it needs more effort to analyse the impact of change in terms of cost, benefits, and risks. *Requests for Change* are concerned with the services that could be seen as projects, i.e. deployment of a video streaming system.
- 2) *Service Request*: this type of service usually affects one individual; it should take only a short time to implement, typically on the same day and takes hours to implement. *Service Requests* are concerned with the maintenance and daily operations of the IT services provided, i.e. reset password or requesting access.

We organized the 24 teams into eight strata based upon the type of service provided by each team, the eight strata are outlined below in Table 2, however, the team names have been omitted to keep the confidentiality of ORGUS.

Within each stratum, we used *Purposive Sampling* (Easterbrook et al. 2008) and identified eight teams as a starting point. The sample included three teams from Product Design and Development, and one team from the remaining strata except financial management and human resources.

The *purposive sampling* was helpful to choose the appropriate teams for the study. The teams were selected based upon several factors, including but not limited to: the importance of the core services they provide, the resource availability, the team willingness to participate in the waste identification initiative, and the number of different workflows performed by the team (the lower the number, the more the preference to include in the study).

Table 2 The decomposition of the teams into strata

Stratum	Description
Financial management	Includes a team of 9 members responsible for providing financial services, such as accounting, budgeting, and procurement.
Human resources management	Includes a team of 7 members responsible for providing human resources services, such as recruitment, payroll, and employees advancement.
Product design and development	Includes 3 teams of 37 members responsible for product development, design, architecture, standards and project management
Infrastructure operations	Includes 4 teams of 47 members responsible for networking and cabling, datacenters, and hardware repairs (laptops, projectors, and other electronic devices)
Service and application hosting	Includes 6 teams of 61 members responsible for security, server and storage upgrades, system administration, and application hosting.
Client services	Includes 3 teams of 32 members responsible for providing helpdesk services, printing services, call center and receptionists.
Onsite services	Includes 4 teams of 41 members responsible for providing support for desk side (not running on the server side) applications and hardware issues requiring onsite visits.
Relationship management	Includes 2 teams of 18 members responsible for marketing and managing client relations, in addition to continuous evaluation and improvements to the existing services through the Information Technology Services Management (ITSM) team.

3.2 Action Research Project Approach

The action research methodology was implemented during this study. However, to increase the level of scientific rigor, we imposed *Cooperative Method Development* developed by Dittrich et al. (2008) that implies implementing the *Action Research Cycle* by its three phases: *Understanding Practice*, *Deliberate Improvements*, and *Implement and Observe Improvements*. The three phases were chronologically implemented as described below:

Phase I Within the *Understanding Practice* phase, the ethnographic method was espoused to better understand the practice, as this phase is considered the base for the second and third phases of the *Action Research Cycle*. The primary investigator was considered *normal participants* rather than *observing participants* (Runeson and Höst 2009; Easterbrook et al. 2008), which helps identifying the gaps between what the people were actually doing and what they said they were doing within ORGUS.

As described in section 2, the successful implementation of Lean initiative requires underlying changes in the organization's culture and habits (Poppendieck and Poppendieck 2003; Petersen and Wohlin 2010); hence, to understand ORGUS' culture, as a starting point, we analysed the ORGUS' policy documents, and in an attempt to understand ORGUS's employees, we reviewed the "personality analysis reports" that were prepared by a third party professional company.

The analysis supported the decision to use a *Double-Loop Learning* approach (Argyris 2004). This approach was used as it helps to detect and to correct the problems by implementing the required changes not only to *action strategies* (the practices of the employees) but also to the human being's values that govern their *theory-in-use* that derives their actions. For more details, see (Argyris 2004).

Support from senior management and buy-in from employees were both necessary to influence the changes to the employees' *theory-in-use* and transform their *action strategies* from *Defensive Reasoning* to *Productive Reasoning* type of employees, which in turn should help in transforming ORGUS to a learning organization where the employees were encourage to detect and correct the problems they create, see (Argyris 2004). This transformation was required to facilitate the implementation of the required changes and encourage employees to detect the wastes and eliminate them.

Phase II During the *Deliberate Improvements* phase, the second phase of the *Action Research Cycle*, the design of possible improvements was carried out in cooperation with the *research participants*, mainly, the team leads and directors. Eight team leads, four directors, two business analysts; one project manager (the primary investigator) and a change manager were involved through the *Deliberate Improvements* process.

Phase III The final phase of *Action Research Cycle* in this project was *Implement and Observe Improvements* where the strategies and recommendations of the *Deliberate Improvements* phase have been accepted and agreed upon to generate commitment to start implementing them (Argyris 2004; Dittrich et al. 2008). In addition, the investigators during this phase collected data about the performance of the organization using auto-generated reports and customers' satisfaction data to be the base for the improvements measurements (see section 5).

To illustrate our action research process, consider the following real example: during the *Understanding Practice* phase, we found that ORGUS had eight different active directories (AD) that had resided in different servers. During the *Deliberate Improvements* phase the recommendation were made to build a central AD, while in the *Implement and Observe Improvements* phase, a project to consolidate the ADs

was implemented successfully and a central AD was built. ORGUS invested a considerable amount of resources to build the central AD, however, on the long run, ORGUS will definitely save time, effort, and cost associated with maintaining different ADs and servers; in addition to the saving on customization and integration required for each application and system that require authentication thorough ADs.

3.3 The Project Approach

A successful Lean initiative usually requires the support of senior management and buy-in from operational level personnel (Dennis 2007). The first step towards getting senior management support was to convince senior management that applying Lean principles in general, and waste identification and elimination in particular, can help ORGUS in providing higher quality products and services in shorter cycle time, while the operational cost can be decreased dramatically.

We had shaped better understanding of the problems faced by ORGUS through observations and informal, friendly discussions with operational level staff. We began each discussion by pointing out the problems we noticed and asked employees for their personal observations. We highlighted the problems we identified and discussed them with the staff and senior management, which encouraged staff to speak up and raise their problems to the surface. This was a key to creating organizational buy-in.

We then had to convince senior management that we understood their problems, and that we have had a comprehensive understanding of Lean principles and the experience on how to apply them. We started by giving a presentation about the Lean philosophy to the Board of Directors. We focused on their viable problems, giving examples of how they might be addressed by applying Lean principles, concepts, and tools, i.e. *Create Value*, *Just-In-Time* and *Kanban*, *Poka-Yoke* and *Built-In Quality*, *Standardization* (Dennis 2007) however, we mainly focused on *Eliminating Wastes*.

The first step in eliminating wastes was ensuring that each team conducted only the tasks that were necessary (value added) for their processes and eliminated all others, unnecessary (non-value adding) tasks. To achieve this goal, the team leads were asked to identify the services they provided, and to mark the ones they believed should be carried out by other teams. In addition, they were asked to suggest which team should undertake these services.

The team leads were then invited to meet and discuss the identified services to reach consensus on who should run which of the services. The preliminary result of this initiative was impressive; we identified approximately 20 duplicate services provided by different teams, more than 100 services provided by the inappropriate team, and about 15 services that should not be provided at all.

During the evaluation of the services under transition, the new service providers were asked for their feedback on how to improve the service quality, reduce the response time (lead time), and reduce cost. The feedbacks from the new service providers were deemed to be insightful; they provided the “fresh pair of eyes” which is often needed to determine wasteful components within existing processes.

As this project was not intended for scientific research only, the use of the technique to represent the value stream was based on several factors, the most important were cost and time, the options to represent the value stream (current and future) were: *Value Stream Mapping (VSM)* (Dennis 2007), *Product Development Value Stream Mapping (PDVSM)* (Morgan and Liker 2006), or SIPOC and Process Flowchart (Przekop 2006).

During the project, the investigators have had the chance to discuss the options with two of the Lean Product Development pioneers, Dr. Jeffery Liker and Mr. Ron Mascitelli, both of them explicitly stated that VSM is a powerful tool for manufacturing but not for IT/Engineering processes as the nature of the value stream in both industries are different. For more details about the differences see (Morgan and Liker 2006).

Dr. Liker mentioned PDVSM, the extended version of VSM developed by Morgan (Morgan and Liker 2006), however, PDVSM is quite complex and requires training that is

costly and unplanned investment for ORGUS at this stage. We quote a response from Dr. Liker “Jim Morgan developed this at Ford and had a lot of time and resources. Generally I have found simpler versions of mapping for IT or PS are better”.

At the end, both Dr. Liker and Mr. Mascitelli commended the idea of using process flowchart as far as it covers the metrics of waste elements, i.e. takt-time, the available production time divided by the customer demand (Dennis 2007), waiting time, cycle time, lead time, and *Non-Value Added* (NVA) versus *Value Added* (VA) time.

We used a straightforward approach to identify the wasteful processes that were “not required”, by simply asking three questions:

- I. Does the activity, or the process increases customer satisfaction?
- II. Does the activity, or the process reduces cost, time, or effort?
- III. Is the activity, or the process mandated by a legal authority? Is it a rule that we need to follow?

If the answers to all of the questions were “No”, we would analyze the process and conduct a process improvement discovery procedure, using the SIPOC that stands for “Supplier, Input, Process, Output, and Customer”. The modelled process is usually represented as a block diagram that also includes several activities (Przekop 2006). An example is shown below in Fig. 1.

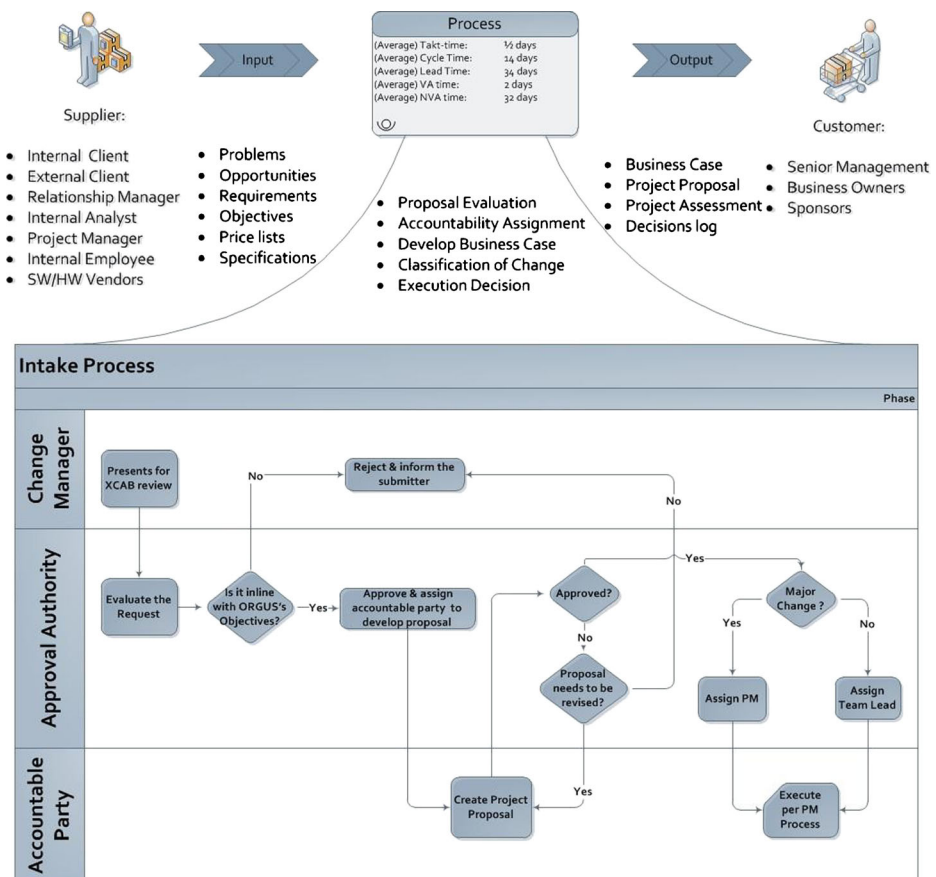


Fig. 1 SIPOC and process flowchart

In the example of the “Project Intake Process” represented in Fig. 1, we used the process flowchart to visualize the block diagram process within SIPOC. We first identified “who” are the suppliers, “what” do they supply, “how” is the process done which is presented using the flowchart, “what” are the outcomes, and “who” are the customers. This identification has given us the details required to pinpoint the potential wastefulness activities in the workflow by confirming the requirements and expected outcomes with the right stakeholders who were identified by the “who” questions.

Processes with high lead and NVA time were broken down and were presented in a greater level of details, going down to the activity level for which we identified more opportunities for improvements. The team leads also discussed and agreed to deadlines for the setup, configuration, and required training that would be necessary to transition the services from one team to another team(s). The transition process and activities are shown in Fig. 2.

As illustrated in Fig. 2, four-stage transition process was used during the transition process. First, we identified the service(s) to transition. The team member who was responsible for running the service was to identify the services that were to be transitioned. The current team

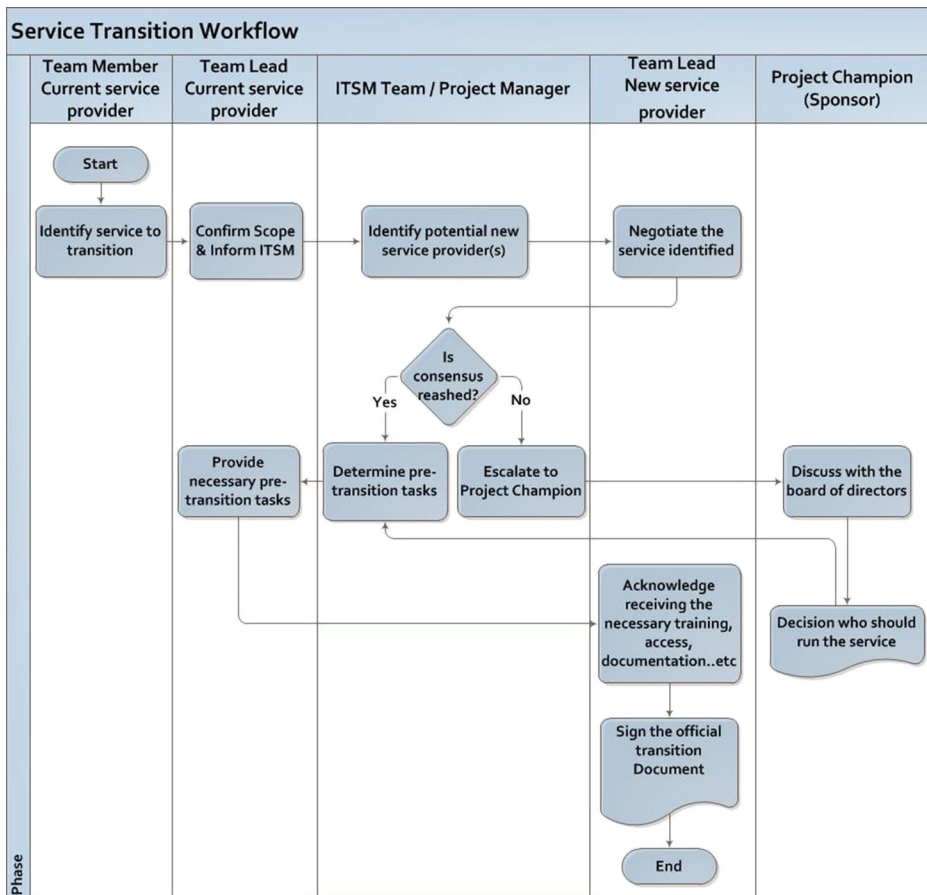


Fig. 2 Service transition workflow

lead—“current service provider”—was to work with the project manager to confirm the scope of the service under transition and inform the IT Service Management (ITSM) team.

Secondly, transitioned services were negotiated and assigned. The ITSM team lead worked with the project manager to identify the potential new service provider and meet with the potential service provider team lead to discuss and negotiate the service to be transitioned. If a consensus was reached, stage three was triggered. Otherwise, and if the newly identified team lead refused to accept the responsibility, the issue was escalated to the project champion—the sponsor—who would discuss the matter with the Board of Directors to decide which team was to accept responsibility for the service.

During the third stage “the transition”, the activities that would guarantee a smooth transition of the services were determined and delivered, including training, configuration, granting privileges, and any other necessary activities to facilitate the transition. At the end of this stage, the services would be transitioned to new teams. A formal sign-off would be obtained during the fourth stage and the service queue would be updated to reflect the changes, and the clients for each service would be informed.

Several challenges arose during the transition process; the most frequent and problematic of these challenges can be summarized as follows:

- Scheduling the meetings; finding the time to put several individuals in one room was very challenging.
- Reaching consensus, as more than 25 % of the identified services were escalated to senior management to make final decisions on them (the total number of services identified was 351 services).
- Building the transition plan and scheduling as the new teams complained about the lack of resources to accept new services.
- The implementation of waste identification, determining, and depicting the value stream were challenging, as none of the employees has the experience or the knowledge to use VSM or PVSIM.
- The communications were very difficult and required tremendous efforts to coordinate.
- The toughest challenge was changing the culture; changing the ORGUS policies to accommodate the changes was not as hard as changing the employees’ behaviours. For example, it took ORGUS’ employees awhile to follow the SPOC policy of not commencing a work without a work-order assigned by the helpdesk.

3.4 Identifying the Waste and Evolving the Model

The identification of waste was an ongoing process conducted throughout the transition process, the classification was developed, and wastes were categorized as they were revealed. During the evaluation of the services under transition, the new service providers contributed their feedback on how to improve the service quality, reduce the response time (lead time), and/or reduce cost.

The feedbacks were usually obtained through face-to-face discussions between the research group and the new service provider and his/her staff, based around specific interview questions. The questions that were asked included:

- 1) If you are given the chance to change the way the service is offered, what would you like to change? Why?
- 2) How would like to implement the change? Why?

- 3) What are the benefits/advantages of implementing the change?
- 4) What are the drawbacks of NOT implementing the change?
- 5) Do we have the skills and experience to implement the change?
- 6) Who will be involved in implementing the change?
- 7) How long does it take? How much does it cost? (Rough estimations)

The feedback from the new service providers were deemed insightful; they provided the “fresh pair of eyes” which is often needed to determine wasteful components within existing processes. The feedbacks saved money and resources for ORGUS as the new service providers identified several services as duplications. These duplications had been identified during the development of the SQM catalogue that specifies only one team responsible for each service.

In an instance, during a follow-up meeting, a new service provider team lead discussed the licensing agreement for an application for which the licensing is based on the number of CPUs. For the licensing purposes, the application provider considers the following:

- Each physical CPU equals four virtual CPUs
- User Acceptance Testing (UAT) CPU should be licensed for half price
- Development (Dev.) CPU is free of charge
- Physical CPU license costs \$30,000

ORGUS was licensed for four physical CPUs. Two CPUs were used for production, one for testing, and the fourth for development. The team lead suggested building a clustered environment of two nodes to increase the availability with four virtual CPUs on each node to increase the performance for a total of eight virtual CPUs. In addition, two virtual CPUs were dedicated for UAT on each node for a total of four virtual CPUs.

The total licensing should come to three physical CPUs in which one of them was dedicated for UAT. The overall results, enhanced availability, enhanced performance, increased customer satisfaction, lower efforts in monitoring the application, and saving of \$45,000 per annum. Organizations need to encourage a culture in which employees continuously identify root causes of wasteful effort and develop strategies to eliminate them. Instead of spending effort, time, and resources to implement correcting actions. Organizations should strive to design and implement either preventive or corrective actions (this is a required to transform employees from type *Model I* to *Model II*), for more details, see (Argyris 2004). Organizations can avoid the effort and cost of correcting actions by eliminating root cause sources of waste, which implies that subsequent wastes are also eliminated.

Wastes that were identified during the transition project in ORGUS are highlighted in this section, organized by the waste classifications of the proposed model that was created and revised during the identification of the wastefulness processes in ORGUS. Some wastes could be categorized under more than one category; however, we identified the waste underneath one classification that was determined to be its root cause.

The 5 *Whys* technique was employed to identify the root-cause (Besterfield 2009), this technique was developed and used by Toyota during the evolution of the TPS (Murugaiah et al. 2010), to illustrate the 5 *Whys* technique, consider the following real example that took place during the project:

1. *Why do the employees in ORGUS implement work-arounds?*
Because they try to find quick fixes for problems.

2. *Why* do the employees in ORGUS try to find quick fixes for problems?
Because they try to avoid waiting for customer's feedback.
3. *Why* do the employees in ORGUS try to avoid waiting for customer's feedback?
Because getting the customer's feedback is time consuming.
4. *Why* is getting customer's feedback time consuming?
Because contacting the customer requires the relationship manager's approval.
5. *Why* does contacting the customer require the relationship manager's approval?
Because it is a company policy!

Obviously, the root-cause of the identified waste was not lack of customer involvement (see Section 3.4.3); the root-cause was centralized decision-making (see Section 3.4.5). The root-cause could be something else if the answers were different (Murugaiah et al. 2010), however, to validate the analysis of the root-cause, we brainstormed the questions with 15 different participants in three different meetings.

The brainstorming sessions resulted in nine participants (60 %) providing answers that led to the identified root-cause, while six participants (40 %) answered the questions in ways that led to lack of customer involvement as a result of the absence of well-established customer involvement strategies.

In the proposed categorization, we identified the waste mentioned above under the lack of customer involvement as this particular waste could be found in decentralized organizations, however, we referred to the centralized-decision making as the root-cause to maintain the research honesty and integrity.

3.4.1 Waste Classification (a): Gold Plating

Gold plating in software engineering refers to the development of unnecessary features or refinements. Gold plating is considered to be an extremely poor project management practice: it results in the addition of features that are not strictly within scope, or “scope creep” (Project Management Institute 2008) that leads to cost overruns.

Developers are fascinated by new technology and keen to develop new features (“bells and whistles”) that, in their view, will enhance the software product; as well, developers often assume that their customers will be happy to see additional features in their products. Customers, however, might be disappointed by the results (McConnell 1996). In addition, the effort required to design, implement, test, document and support features that were not required lengthens the schedule and incurs additional cost (McConnell 1996) and must be seen as a waste. *Gold plating* in its generic and broader definition may include the provision of redundant information, using unnecessary tools or technologies, or providing an intricate solution rather than the simplest one.

The waste and elimination strategies that we identified as part of the “Gold Plating” waste classification are:

“Unnecessary tools, technologies, or methodologies”

Likewise identifying the wasteful processes, we used the same approach to identify required tools, technologies, or methodologies that were “not required”, by simply asking three questions:

- i. Does the use of the tool, technology, or methodology increase customer satisfaction?
- ii. Does the use of the tool, technology, or methodology reduce cost, time, or effort?

iii. Does a legal authority mandate the use of the tool, technology, or methodology?

If the answers to all of the questions were “No”, we would analyze the process to support the recommendations to stop or minimize using the tool, technology, or methodology. Before evaluating the feasibility of the recommendations, we reported our findings to senior management for their review and approval. The ITSM team led the feasibility analysis phase, during which the effects of the recommended process changes were assessed in terms of cost, risk, and benefits.

A MoSCoW matrix is a prioritization technique. MoSCoW categories are: M-Must do it, S-Should do it, C-Could do it, W-Won’t do it (International Institute of Business Analysis 2009). Senior management reviewed the resulting proposals during their regular biweekly meetings, and used a MoSCoW matrix to prioritize the proposals. The highest priority proposals were placed in the appropriate queue for implementation.

“Producing and distributing reports that are not required, and providing more information than is necessary”

Providing more information than is necessary generates “time and effort” waste for both the report’s author and its reader(s). An analysis of the reporting scheme in ORGUS revealed that, on average, 60 % of the recipients did not have any significant interest in the reported item; moreover, these recipients did not know why they were receiving the reports. On average, 70 % of the recipients indicated that the reports provided useless or supplementary information that the recipients said they did not need to know.

The analysis of the reporting scheme was conducted through semi-structured interviews that included 20 directors and team leads. The questions were asked during the interviews included:

- 1) What reports you usually receive? Why?
- 2) What report(s) you should have received? Why?
- 3) For the report(s) you receive, do they provide sufficient details?
 - a. If not, what are the missing details?
- 4) Do they provide more details than you need to know?
 - a. If yes, what are the extra details?
- 5) What would you like to see in status/performance/weekly/incident report?
- 6) What is your preference for report presentation and format?
 - a. Presentation (oral) vs. written? Why?
 - b. Word, PDF, PowerPoint, Prezi...etc.? Why?
 - c. Graphs / charts vs. written description? Why?
- 7) For the current reports, what (you think) should be reviewed and revised? What changes would you like to see? Why?
- 8) How often should you receive each report?
- 9) How do you receive the report? Email attachment, hardcopy...etc.? How would you like to receive the report?
- 10) How much time you spend on reading reports? (Per week)
- 11) How much time (you think) you should spend reading reports?
- 12) On scale from 0 to 10 (0 is min and 10 is max), how do you evaluate the overall reporting system, format, details, presentation, distribution...etc.?

13) Do you have any question, comment, or suggestion?

The solution was to create a report template that contained the information that the recipients had deemed to be necessary based upon the requirements collected during the analysis phase. The new report template was revised based upon the comments and the feedback that had been provided from the directors and team leads. After 4 months of using the new template, we asked the question number 12 again. The overall evaluation was increased from 3.65 to 8.45.

“Unnecessary cosmetic functionality designed and developed into software”

New technology tends to fascinate some developers, and its use can lead to the development of unnecessary new features or refinements without corresponding business requirements. To limit the resulting waste of effort, time, and resources, we introduced a Kanban board (Middleton 2012); items to be developed were to be clearly identified and visually represented on this board. In addition, we introduced the concept of “stand-up” meetings to ensure the team was focused on delivering only needed functionality, thereby reducing feedback cycle to one working day.

The research investigators were closely involved during the setup of Kanban and the early stage of the stand-up meetings; they attended the meetings for 4 weeks and provided guidance and feedback on how to run the meetings. Based upon the feedback from the Software Development team lead, Kanban has helped keeping the developers focused on the features they had to develop through the use of the Kanban board.

“Striving to design perfect systems, rather than designing systems that simply are adequate”

Our analysis showed that the system designers and architects in the ORGUS aimed to develop perfect system designs and architectures, rather than the most simple and “sufficient” designs and architectures. These employees tended to work in isolation, without taking into consideration other external factors, such as budget availability, lack of resources, and lack of required skills.

Initially, we tried to solve this problem by applying the *Quality by Design* philosophy through which design flaws and pitfalls are avoided by identifying the risks and imitations of the design (Besterfield 2009). However, we noticed that the problem was, in fact, more related to the designers’ and architects’ mindsets: we reviewed these employees’ “personality analysis reports”, we found that 100 % of the system designers and architects in ORGUS were categorized as perfectionists. Therefore, we needed to change the values that governs the *theory-in-use* through implementing the *double-Loop Learning* (Argyris 2004) and promoted the idea of inquiring. Daily stand-up meetings formed a solid ground to build a collaborative environment and reminded the designers to ask themselves: Why do we develop it this way? Can we develop a simpler design?

3.4.2 Waste Classification (B): Over-specifications

Developers and analysts sometimes define requirements and specifications that are not strictly required. For example, performance is stated as a requirement more often than it needs to be, and this can unnecessarily lengthen deployment schedules (McConnell 1996). Over-specification wastes may also cause gold plating wastes.

The waste and elimination strategies that we identified as part of the “Over-specifications” waste classification are:

“Excessive analysis that does not support decision-making”

Analysis requires a large investment of time and effort. We observed that the analysts in ORGUS focused on specific problems and began “digging deep” into these problems during the early stages of the project lifecycle. Consequently, the analysis process produced a large amount of data that needed further analysis, leading to what sometimes became an almost infinite cycle to “gather information and analyze further”.

For example, in a project to implement a centralized secure printing solution, the business analyst had found that the clients were using different printing station models, hence, he started “digging-deep” to explore how the printing solution could be integrated with these different models, which led to more analysis of this specific requirement. The business analysis should have reported the findings and left the solutions architects and designers to look into how these pieces should be integrated together.

The elimination strategy for this identified waste was straightforward; however, time was needed to implement it correctly and to ensure consistent execution. First, we suggested that the team focus only on what is needed at the specific stage of the project lifecycle. In this case, we recommended that a high-level analysis to identify the problems be undertaken within predefined time frame, without detailed analysis of any one problem.

The solution was realized by emphasizing the use of a template that we had prepared earlier based upon senior management’s requirements. We used the same interview protocol and questions to analyze the reporting scheme as described in Section 3.4.1; each section of the new requirements template contains a precise description of what should be included in that section, which encouraged the analysts to provide only the information needed and not go beyond these requirements, thereby avoiding unnecessary work. Finally, we monitored the analysts’ results to ensure they were producing the desired outcomes and adjusting their plans during the stand-up meetings.

“Searching for information that is difficult to find”

Searching for information that is difficult to find is a project bottleneck that contributes to “waiting waste”. In ORGUS, we set out a “half day” rule for searching for information, if the search to find the information takes more than half a day of effort, the information is marked as difficult to find. Having identified this waste in the ORGUS, we proposed two options:

- 1) If “enough” information existed to complete the task in hand, proceed further.
- 2) If more information was required, the analysts should work with subject matter experts to state all assumptions and document the assumptions as potential risks. We emphasized that assumptions should be seen as potential source of waste, and should only be stated based upon a consultation with subject matter experts and, if and only if, the information is difficult to find.

“Overdesign of software or application”

Generally, in the “real world”, the time between preliminarily system design and the final product does not shrink if more analysis takes place upfront. Overdesign wastes effort, time and resources, and may result in a design that does not fulfill the customer’s requirements. The

application of customer feedback model allows for the validation of the design and guarantees that the focus is on what really matters to the customer. This recommendation was just left in the *Internal Implementation* stage. For the complete list of possible solutions to eliminate wastes and their implementation status in ORGUS based upon resources availability, see Table 4.

3.4.3 Waste Classification (C): Lack of Customer Involvement and Inappropriate Assumptions

In many IT organizations, making assumptions is a habit that results from a lack of customer involvement. Thinking on the customer's behalf is not uncommon in IT organizations. As well, developers make assumptions about the eventual production environment, and although these assumptions may be relevant, for example, for a developer's workstation, they may be less relevant or completely inappropriate in a users' "production" environment (Humble and Farley 2011).

Inappropriate assumptions can lead to the creation of new defects, which are hard to discover and fix, especially after the service, or system has been deployed into a production environment (Humble and Farley 2011). Since the probability of introducing defects is high, end users will be significantly affected which leads, in turn, to low customer satisfaction. The waste and elimination strategies that we identified as part of the "Lack of Customer Involvement and Inappropriate Assumptions" classification are:

"Misunderstanding of user requirements and developing software based on expectations of what the client wants"

As part of this research study, we reviewed some previously completed projects carried out by ORGUS, focusing our review on those projects that had not met the needs or that had exceeded the time and/or budget. The common team behaviour that we identified in these projects was that of "thinking on the customer's behalf". Failure to spend "sufficient" time in the requirements gathering phase, and limited communication with end users resulted in project failure.

Hiring competent business analysts was a relatively simple solution to eliminate the above-mentioned waste. ORGUS hired a competent business analyst who subsequently made a noteworthy contribution by eliciting more than 40 major functional, non-functional and transitional requirements, successfully delivering what the customers needed by focusing on requirements elicitation techniques. One large project was put on hold for about 30 days to gather customers' requirements: as a result, the project's scope, duration, cost, and completion criteria changed significantly. The major drawback to this solution was the cost associated with the hiring; however, senior management considered the "cost of quality" that would have resulted from not implementing the recommended solution and decided to make the investment.

"Inflexible and lengthy approval process that encourages work-arounds"

Different reasons may lead employees to solve problems by implementing work-arounds, some of these reasons are, unavailability of the customer to answer questions or to provide feedback and the desire to finish the task in hand as soon as possible, subsequently, they try to find a quick fix which most likely will be a work-around. Generally speaking, eliminating a root cause waste will eliminate subsequent waste. However, the reverse is not true:

elimination of “down-stream” waste does not necessarily result in eliminating the root cause waste. An inflexible and lengthy approval process that encourages work-arounds (subsequent waste) is the direct result of centralized decision-making (root cause waste) that has been identified as a stand-alone classification, to eliminate this waste, ORGUS has changed some of its policies to empower the employees to overpass some approval processes, which reduced the waiting time and increased the employees’ morale.

An example of the policies that ORGUS changed was related to procurement for which the requester had to go through four approval levels 1) Team Lead or Project Manager, 2) Direct manager/supervisor of the team lead or project manager, 3) Director of the unit/department, and 4) Procurement Manager. ORGUS has eliminated the second and third approval steps and changed the policy to place the purchase order if the team lead or the project manager, and the procurement manager have approved it.

“Work performed earlier than needed because of the absence of prioritization and dependencies”

Due to the very large number of projects that the IT department needed to manage, as well as the new project proposals arriving every day for review, a systematic approach to evaluate the importance, priority, and dependencies between projects was necessary. We developed a MoSCoW matrix to assign project priorities based on factors that ORGUS considered to be important, as shown in Fig. 3.

The use of MoSCoW prioritization helped senior management to decide which projects to initiate and which ones to put on hold. As well, the matrix helped to clarify and manage the dependencies between projects, and prioritizing the projects contributed to fact-based decisions and enhanced resource management effectiveness and efficiency.

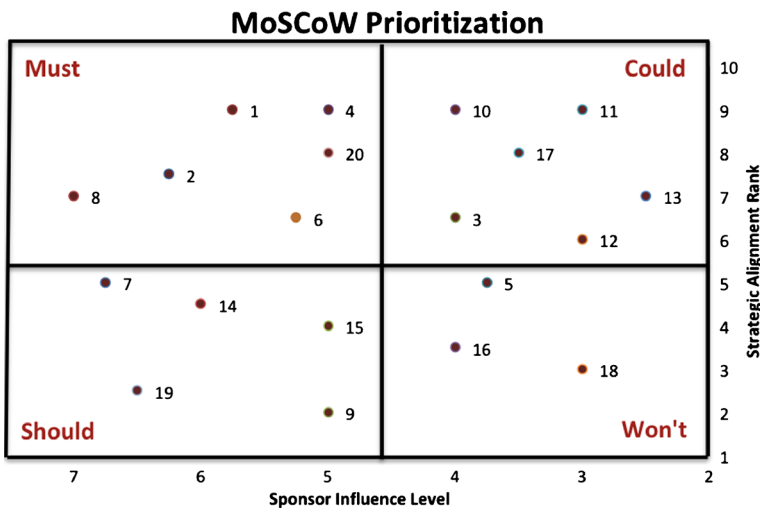


Fig. 3 MoSCoW prioritization matrix

“Lost information, thus, developer starts to make assumptions”

We noticed that each team in the IT Department had its own data repository; as well, very few teams had a backup mechanism in place for their data, while the others had multiple repositories for backup. We found that only one team used the department’s version control system. Ironically, the organization has an enterprise content management system (ECMS) and a collaborative shared document environment.

To eliminate the waste associated with lost information, we needed to eliminate its root cause. Our proposed solution was to increase awareness of the existing and available backup, version control, and document management systems and to mandate and regulate their use.

“Limited access to the customer”

The reasons behind the limited access to client were varied; however, the most prevalent reason was the use of improvised or ad hoc customer access models. We recommended the adoption of more formal customer involvement methodologies, depending on the need for the customer’s feedback. These involvement methods ranged from daily customer involvement – that is, customer as part of the team, using the tactic of the Extreme Programming (XP) methodology – to periodic feedback only, using prototyping methodologies (for more details, see (Martin et al. 2009)).

This recommendation has been just left in the *Internal Implementation* stage in which neither commitment nor approval has been given toward implementing the recommendations. For the complete list of possible solutions to eliminate wastes and their implementation status in ORGUS based upon resources availability, see table 4.

“Onsite visits to solve problems that could be solved remotely”

Onsite visits to resolve problems are costly and time-consuming. We encouraged the use of monitoring systems and remote access software packages to undertake initial problem investigations as a simple strategy to eliminate this waste.

3.4.4 Waste Classification (D): Double Handling / Duplicate Processes

In today’s business environment, Internet connection failures, email problems, and non-responding systems are among the biggest nightmares for business leaders. Non-IT leaders panic when they experience these IT-related problems and often escalate this panic randomly. In turn, IT personnel react immediately, especially if the person who is experiencing the problem is a senior manager or a lead customer. In many cases, more than one IT technician—sometimes an entire team—begins working to solve the problem. This duplicated effort magnifies the cost to solve the problem, and the work done by one person or team to fix the problem might conflict with the work that the other person or team is doing to fix the problem, thereby creating more problems.

In large IT organizations or IT departments providing services to large organizations, it is usually difficult to completely eliminate these duplicate processes. Reducing these duplicated processes, however, is possible by analyzing the services provided using “Responsible, Accountable, Consult, and Inform” (RACI) matrix, illustrated in Fig. 4.

Process Name / Description:

Cloud Printing Project

Created On:

12-01-29

Revision:

12-04-01

Created by:

Owen Jordan (PMO Mgr), Osama Al Baik (Project Mgr), James Miller (Project Sponsor)

Activity / Task	Role	PMO Mgr	Sponsor	Technical Lead	Technician	Project Mgr
Identify a minimum of three printing solution providers (Vendor)		C	-	-	-	R
Arrange for vendor visits and quotes		I	-	-	-	R
Review quotes and references, make vendor selection		A	I	I	-	R
Review and finalize contract		I	I	-	-	R
Communicate project to stakeholders, make sure all users are aware of the go live date!		I	I	R	I	I
Provide access to users to be able to print by June 15		I	-	A	R	I
Oversee the project, ensure it is completed on time		A	I	I	-	R

R = Responsible, A = Accountable, C = Consulted, I = Informed

Fig. 4 RACI matrix [template: racichart.org]

The waste and elimination strategies that we identified as part of the “Double Handling / Duplicate Processes” classification are:

“Providing the same services by more than one team”

We identified duplication, and in particular, multiple teams were offering the same services, as one of the major wastes in the ORGUS. The clients typically approached different people in different teams to report a problem. Because of the high volume of requests received daily and limited communication between different teams, it was very difficult to identify who was working on a particular problem. After spending some time to identify possible solution, the study team concluded that the organization should implement a “Single Point of Contact” (SPOC) and develop a service queue map that would be used to send the problem to the appropriate team once it was received. As a result of this recommendation, the organization initiated two new projects, namely, a SPOC project and a “Service Queue Mapping” (SQM) project; both have been established as stand-alone and separate improvement projects in the organization.

SQM aimed to prepare a service catalogue to be used by the SPOC team to route and dispatch the *Service Request* or the *Request for Change* to the appropriate team. The service catalogue included the service name, description, technology used, list of supported client, escalation levels, and the responsible team to solve the problems.

During the identification of a responsible team for each service, we noticed that several teams had claimed responsibility for providing 21 duplicate services, the team leads were invited to make sure that the services provided were identical, then a transition process was initiated to identify first and second level of support.

The SPOC and SQM has realised several improvements, including, but not limited to:

- 1) Increased customer satisfaction;
- 2) Reduced response time, as it has become easier to route the reported issue to the appropriate team; and
- 3) Increased productivity and reduced cost, as one employee is guaranteed to work on the issue rather than having multiple employees working on the same issue. This has been realised through assigning work-orders by SPOC and creating a policy in ORGUS that no work should be started without an assigned work-order.

“Having redundant Data in the same or different systems”

One of the many problems associated with storing duplicate data is the amount of effort and time needed to maintain and update the same data in multiple locations. Within the same system, the problem of duplicated data can be simply addressed by normalizing the data in the database (Xingjian 1992). However, if the data is duplicated in different systems, the solution lies in establishing a central data repository. The situation was fully described as an example in the section 3.

“Multitasking, in which people move between incomplete projects because of the absence of project prioritization”

We noticed that employees stopped undertaking their daily operational tasks to undertake project related tasks when a project manager pushed them to work on a project with a deadline; the employees began missing deadlines of their operational tasks because of this movement between tasks. As mentioned previously, a waste elimination strategy can eliminate more than one waste at a time. In fact, waste elimination strategies themselves should be developed based upon the Lean principles. MoSCoW prioritization is one good example, which can be used to eliminate several wastes at once, and can be utilized to prevent the organization from creating other wastes, the waste resulting from multi-tasking could be eliminated by prioritizing the work on a team level using a MoSCoW prioritization matrix.

“Repetitive, unnecessary, and ineffective meetings”

Avoiding unnecessary meetings is a time management “best practice” (Walsh 2008). As well, the best strategy to use to avoid unnecessary meetings is to not schedule a meeting in the first place!

A simple estimate of the time spent in meetings at ORGUS revealed a rather large amount of money was being spent on ineffective meetings. Top management in the organization was fully aware of this problem. We identified several reasons why repetitive and ineffective meetings were scheduled and held. These included: lack of an agenda, ineffective meeting facilitation, lack of a meeting review and conclusion summary, and lack of formal adjournment.

The investigators reviewed and analysed the documentations of around 200 meetings to assess their effectiveness. The assessment was based upon several factors that were formed as questions, which are listed below

- 1) Did the meeting have an agenda?
- 2) Did the facilitator distribute the agenda prior to the meeting? If yes, how long before the meeting?
- 3) Did the meeting start and finish on time?
- 4) Did the agenda items have specific time limits?
- 5) Did the agenda clearly define the meeting objectives?
- 6) Did the facilitator assign a meeting scribe? If not, who was documenting the minutes?
- 7) Did the facilitator run the meeting according to the agenda?
- 8) Did the facilitator give time for the attendees to express their thoughts? Did the facilitator/scribe take notes and document these thoughts?
- 9) Did the facilitator review and summarize the meeting outcomes and action items (if any) at the end of the meeting?
 - a. Did the facilitator follow up on action items (if any)?
- 10) Did the facilitator/scribe distribute the meeting minutes? If yes, how long after the meeting?
- 11) Did the meeting achieve its objectives?
 - a. If not, what were the obstacles that prevented the team from achieving the objectives?
 - b. What were the steps taken to remove the obstacles?

The strategy we recommended to minimize the organization's meeting waste was to develop and implement meeting rules. These rules included: defining the meeting's goal; sending invitations, along with a clear agenda, at least two business days in advance; defining time limits for discussion; effective meeting facilitation by engaging attendees and encouraging them to participate; ensuring that any important off-track points should be noted and "parked" for further discussion in other meeting(s); writing clear meeting minutes; identifying action items with specific due dates and follow-up plans; and finally, conducting post-meeting evaluations and reviews. The ORGUS established a campaign to emphasize the importance of effective meeting management, and senior management highlighted the rules to the employees at periodic department meetings.

The investigators had been involved in about 100 meetings related to the waste identification and elimination project, where they used the rules mentioned above. In addition, 3 months after applying the rules, the investigators reviewed the documentations of around 200 meetings. The quality of the meetings results and documentations were dramatically improved.

"Troubleshooting to solve single problem instead of root causes"

Conducting root cause analysis takes more time and effort than simply addressing and resolving a single problem (Ammerman 1998); however, the summation of time required to troubleshoot and resolve each single problem that results from the same root cause is much greater in the long term. The elimination strategy for such a waste must be Lean as well, that is, root cause analysis must be employed. In ORGUS, we made it clear to the teams that, by spending more time to analyze the problem and implementing the appropriate corrective action(s), the cost in the long run would be much cheaper and better than finding workarounds to solve each problem individually.

“Focusing on team optimization not overall departmental performance”

Team optimization processes can improve the speed and quality of a team’s performance; however, such optimization may result in the team missing the “bigger picture” (Bellingham 2004) and not having the necessary information or knowledge of the organization’s long-term goals. As well, teams should not work in isolation to optimize their performance, since organizations cannot produce more than the capacity of their bottleneck operations (Goldratt and Cox 1992).

The optimized performance and increased production speed of any one team is irrelevant if the input operation to that team is a bottleneck. Therefore, optimization should be carried out on the level of the organization as a whole, rather than at the individual team level.

Senior management took a significant step forward by ensuring that the organizational long-term goals and strategy were transparent to all employees. As well, management adopted a waterfall approach to communicate strategic decisions and plans to all employees on a weekly basis. The organization also implemented an open-door policy so that upper management could receive input directly from operational-level employees. All of these changes to the *organizational defensive routines* (Argyris 2004) contributed in shifting the focus in ORGUS from individual team optimization, to the organization overall optimization.

3.4.5 Waste Classification (E): Centralized Decision Making

Under a centralized decision-making model, senior management makes all of the decisions and lower organizational levels implement these decisions (Bhagat and Steers 2009). Any decisions that are made by lower-level employees are subject to the approval of senior management.

Centralized decision-making results in a longer lead-time to implement decisions. The approval process may take longer than necessary because, in many cases, senior management will require more information in order to evaluate the situation before making a decision (Bhagat and Steers 2009). By the time senior management evaluates the situation and makes or approves a decision, the information that was provided as background might be obsolete; this is true especially for time-sensitive decisions.

Centralized decision-making therefore leads to waste and, in turn, creates other types of waste, most noticeably, waiting waste. The best strategy for eliminating this waste is to implement a decentralized decision-making model. Senior management should establish the strategic goals for the organization and make key decisions, but trust and empower their employees to take the actions and make the decisions that are necessary to achieve these goals.

The waste and elimination strategies that we identified as part of the “Centralized Decision Making” classification are:

“Unnecessary approval processes and unclear responsibility and authority”

The ORGUS had undertaken serious actions to restructure its business processes and this effort resulted in major organizational changes. Senior management felt that they needed to retain decision-making authority, especially during the early stages of the restructuring process. As time passed, the organizational structure became more robust and the responsibilities, accountabilities, and authorities in the organization became clear using the RACI matrix. This enabled the organization to eliminate unnecessary approval processes.

“Complex reporting systems that add extra effort without adding value”

Employees in ORGUS were required to prepare different reports for different purposes. As mentioned previously, we created report template based upon stakeholders’ requirements. The project management team experienced additional types of reporting problems that created extra overhead for the projects. For example, project stakeholders asked project managers to create different reports, such as, reports for immediate managers, reports for project’s sponsors, and summary reports for senior management. On average, the time to prepare these reports, based on running six projects simultaneously, was estimated at about 25 % of the project manager’s time.

We considered the time required to prepare these different reports as pure waste since they did not add any value from the customer’s point of view. We recommended an automated reporting mechanism using an MS Excel spreadsheet. Project managers would complete a weekly report template by entering data into the spreadsheet; the different reports would then be populated and produced from this data.

“Making decisions that lock in resources too early and produce inflexibility”

Poppendieck and Poppendieck (2003) proposed, *“Decide as late as possible”* as one of the Lean software principles. Some people confuse planning and commitment (Schyff 2011), planning is required; however, flexibility in planning is desired. As well, effective resource management distinguishes the “good managers” (Bellingham 2004).

Project Management Institute (2008) has recommended the adoption of a matrix organizational structure to help project managers, in collaboration with functional managers/team leads, to plan and build a project schedule according to resource availability. As well, we recommended the use of MoSCoW prioritization matrix, which can be very useful technique to negotiate the availability of required resources for prioritized projects so that these can be delivered on time.

“Lengthy approval processes that discourage innovation and creativity”

As part of its initiative toward the change, ORGUS has applied ITIL (Information Technology Infrastructure Library) best practices to improve productivity, improve customer satisfaction, reduce cost, and improve the IT services themselves. Adopting ITIL implies adopting its set of approved methods (Addy 2007), thereby, eliminating lengthy approval processes, for example, ORGUS has pre-approved a total of 73 standard changes that do not need to go through the approval process. Based on the reports generated from the ticking tool, on average, ORGUS receives a total of 200 standards changes on a daily basis; eliminating the review and approval process showed a saving of around \$235,000 per annum and reduced the waiting time of 2 working days on average.

“Lack of delegation to empower the employees resulting in technical decisions made by non-technical people”

Successful leaders coach, rather than direct, their subordinates (Besterfield 2009). Employee empowerment is one of the Lean best practices (Womack 2007) that can save time and

minimize rework; moreover, empowered employees are more confident and loyal (Womack 2007). Employees are usually more committed to decisions they either make themselves or that they participate in making (Bhagat and Steers 2009).

We explained to ORGUS that the employees who have the largest capacity to add value are not necessary those who are at the top of the organization hierarchy (Bellingham 2004). Rather, the employees who add the most value are usually those who are more intelligent; at the same time, those are the employees who are typically not easy to manage (Younker 2008).

“Smart” employees do not usually accept the superficial value of a decision made by management; this explains why it is necessary to get their buy-in to decisions. These employees are not impressed with titles; they tend to understand the rational behind the decision and believe in expertise rather than simply accepting a decision because of the authority level of the decision-maker (Harvard Business Review 2011).

The project management team was experiencing overhead problem of being involved in every initiative tackled by ORGUS, senior management seemed to see each initiative as a project, while in fact, the project management experts in ORGUS had a different definition of what to consider a project. Project managers were expected to be involved very early in the process and spend time developing project plans for *Request for Change* (for more details see section 3.1) that was still “not approved”. In cooperation with the project management group, we developed a “Project Intake Process” to define what should be considered a project. The intake process is shown below in Fig. 5.

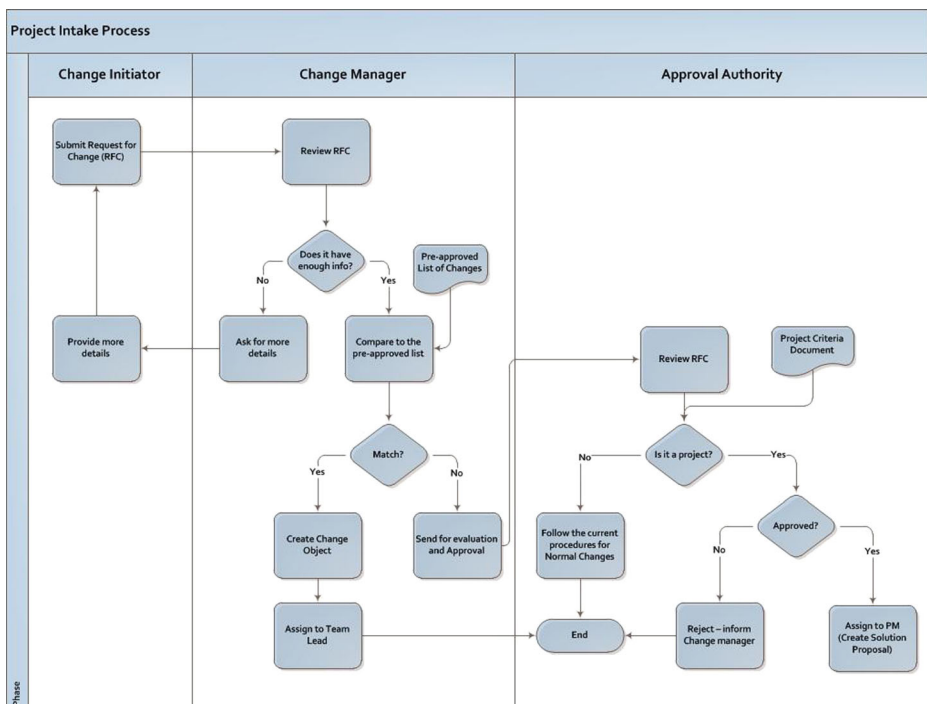


Fig. 5 Project intake process

Waiting waste in an IT environment is similar to waiting waste identified in manufacturing environments. The nature of the waiting waste, however, is different. Waiting waste in IT occurs when an employee has to wait for work to be released from the predecessor operation (Dennis 2007). The main causes of the waiting waste in IT are: waiting for approval on action items (for example, as a result of centralized decision making), fixing

In many cases, IT personnel will try to avoid waiting—not because it is wasteful, but because it is human nature. For example, clients who are waiting in a bank queue will watch the clock every second while they are standing in the queue. However, they will not watch the clock while they are being served, regardless of the time the service takes or the number of other customers waiting in the queue.

3.4.6 Waste Classification (F): Waiting

Waiting waste in an IT environment is similar to waiting waste identified in manufacturing environments. The nature of the waiting waste, however, is different. Waiting waste in IT occurs when an employee has to wait for work to be released from the predecessor operation (Dennis 2007). The main causes of the waiting waste in IT are: waiting for approval on action items (for example, as a result of centralized decision making), fixing defects, or waiting for customer feedback.

In many cases, IT personnel will try to avoid waiting—not because it is wasteful, but because it is human nature. For example, clients who are waiting in a bank queue will watch the clock every second while they are standing in the queue. However, they will not watch the clock while they are being served, regardless of the time the service takes or the number of other customers waiting in the queue.

In order to avoid waiting, IT personnel would initiate an avoidance strategy and start working based on assumptions. These assumptions may not be correct, and may produce other types of waste such as the “inappropriate assumptions” waste discussed earlier in this paper.

The waste and elimination strategies that we identified as part of the “Waiting” waste classification are:

“Waiting for review and approval on action items”

The implementation of ITIL in ORGUS resulted in a faster approval process for the action items and corresponding change requests that were on the pre-approved list. However, major action items that triggered major *Request for Change* (for more details, see section 4.1) still needed to be reviewed and approved. The solution for the waste that is produced by waiting for review and approval of the major action items is embedded within the ITIL best practices and involves the implementation of a *Change Advisory Board* (CAB) that is responsible for reviewing and approving these major changes (Addy 2007). This review and approval process could have a “time box” of 24 h to minimize the waiting time. For more details about improvement resulted from introducing “time box”, See (section 5).

“Waiting for information that is needed to complete a task”

The above waste is an excellent example of a “push versus pull” system. Lean supports the pull system as it creates a continuous flow (Jaisingh 2012) and there is no need to wait to get

the information that is needed to get the job done. Using the knowledge management system to store and access shared documents and project workspaces, employees in the ORGUS have the ability to get up-to-date information at any time, pulling the information that they need from this repository whenever it is needed reduced the waiting time by 2-3 business days based on a survey with 23 participants.

“Using shared resources; waiting to assign specialists to accomplish tasks”

Collaborative decision-making enables better resource-management and should solve the issue of waste related to using shared resources or waiting for specialists to be assigned. The investigators held 3 different focus groups in three separate venues with 23 participants, included 7 directors, 8 team leads, and 8 project managers. The separation of groups offered a more convenient environment for the participants to express their thoughts, and provided a freedom to speak up without the presence of their supervisors. The investigators in cooperation with the participants identified three main strategies to eliminate the waste of waiting for shared resources: 1) specialists should be scheduled in advanced, 2) slack time should be considered, and 3) contingency plans should be developed.

“Developer waits tester to finish before starting work on the next piece of code”

Lean suggests the use of WIP to create a continuous flow and minimize idle time (Tapping 2002). In order to effectively manage their WIP, developers in ORGUS have used a Kanban board to visualize the work items’ progress and continue to work on the items that have been assigned to, within the WIP limit.

When a testing engineer pulls a work item for testing, the developer should not be idle and waiting for the testing results. Once the testing is done, and if further coding is required, the testing engineer should add the work item to the Kanban backlog with an assigned priority. The developer, according to the work item’s priority and the WIP limit, either puts one of the current work items on hold and pulls the returned one from the backlog, or just finishes the current work item currently “in hand” and pulls the returned work item by the testing engineer on completion of the current work item (Boeg 2011).

“Waiting for a physical signature”

Waiting time for a paper-based signature is pure waste; the elimination strategy for this waste was identified to use a digital signature based upon a survey with 11 participants included 7 directors, 3 managers, and 2 executive directors. The use of digital signatures was proposed to senior management at ORGUS; by the time this paper was written, the investigation was underway to identify and select a digital signature technology and online signature service provider.

“Waiting to identify the team for providing problem resolution”

Identifying the team that is responsible for resolving a specific problem is no longer an issue for ORGUS. As mentioned earlier in this paper, ORGUS has initiated

SQM system. This system enabled employees to search for the responsible team, by the service name, by the technology used, and/or by part of the service description.

“Clients waiting for their calls to be transferred to the appropriate team”

Clients of the IT department in the ORGUS use a variety of communication channels for reporting problems or for making information inquiries; these channels include email, online chatting, an online ticketing system, and telephone. Regardless of the communication channel, a “ticket” is to be created and assigned.

The use of email, chat, and an online ticketing system is much faster and more efficient than using the telephone for these requests, for obvious reasons. The telephone is the only communication channel that requires someone to be dedicated to the client as part of the communication task; as well, it is the only channel that requires the client to wait idly (i.e. on the telephone) while waiting to get an answer.

We recommended an automated call messaging system to eliminate the telephone waiting time and to minimize effort and cost; this system uses automated messages and voice-to-text technology to create a “ticket”. This is an example of where the recommendation was made during the *Deliberate Improvements* phase and left on the *Internal Implementation* stage as no actual effort was made to put the recommendation into practice and hence moving to the *Implement and Observe Improvements* phase to measure and validate the result of the *External Implementation* stage.

3.4.7 Waste Classification (G): Deferred Verification and Validation

In the context of a system development lifecycle, deferring verification and validation (V and V) activities means that the V and V are expected to be carried out in later stages of the project. In the waterfall development methodology, this is called the “big-bang” approach. As can be seen in Fig. 6 below, the cost of fixing defects in later stages of the project increases dramatically as a project progresses (Watkins 2009; Humble and Farley 2011)

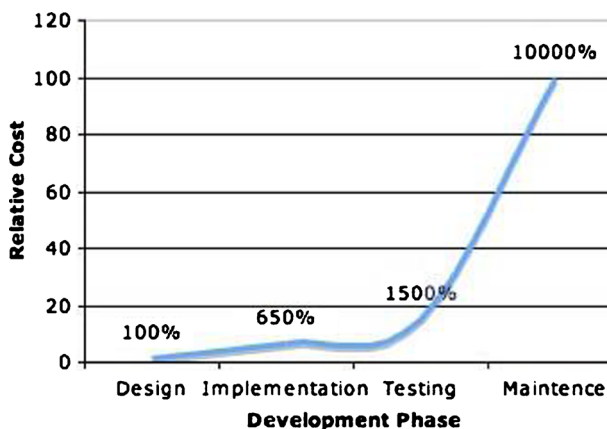


Fig. 6 Relative cost to fix defects [IBM research]

The waste and elimination strategies that we identified as part of the “Deferred V and V” waste classification are:

“Lack of system integration between legacy systems (Waterfall instead of Agile)”

The best solution to reduce the risk of project failures from inadequate V and V is to employ “Continuous Integration” (CI) that integrates the verified features into the existing system continuously. Starting V and V in the early stages of the project lifecycle guarantees higher quality and a lower cost to fix bugs than doing the V and V in later stages. Continuously integrating the tested features into the system eliminates the time, effort, and cost required for integration testing (Watkins 2009). As well, continuous integration helps in validating design assumptions and reducing the feedback cycles.

The “big-bang” approach of the waterfall methodology has been proven to be the most costly in terms of fixing defects (Kim and Pries 2011). Agile techniques in software development reduce the cost of fixing defects significantly by aiming to reveal and fix the defect during its development (Watkins 2009). In summary, waiting until the later stages of development to test the entire development at once is extremely wasteful.

“Excessive standards to follow that encourage avoiding testing”

We identified the use of checklists and coding standards as one of the main root causes of waste in the ORGUS. The IT employees were essentially unwilling to follow any standards written by others. Generally speaking, people prefer to have options; they do not like the fact that they have no other option except to work according to the prescribed standards.

The research investigators introduced the concept of “self-accessed policies” for the first time in a presentation that we delivered to a different IT organization. The logic behind this concept is that a team is more willing to follow rules that it has established itself. The rules that projects’ team members were asked to establish were to cover three coding aspects: 1) typographical conventions; 2) naming conventions; and 3) structural conventions (Yunker 2008).

“Not enough training on, or awareness of, the importance of testing”

ORGUS delivered a series of training sessions on the change processes associated with, and as part of, the ITIL implementation. The importance of testing was emphasized as one of the primary training topics during these sessions. Major action items—those that create and lead to change—require a test plan amongst other plans, before they can be approved. Based on several interviews with more than 50 participants, the importance of testing has become apparent for 85 % of the participants who became more confident of the features they developed or the services they offered.

“Insufficient funds for testing based on budget underestimation”

As mentioned previously, the testing cost should be considered when estimating the overall project budget. The accuracy of these testing cost estimates will improve with experience. The use of the collaborative decision-making and historical data from similar projects that is available in the shared environment makes it easier to more accurately estimate project costs. As well, applying a bottom-up estimation strategy has proven to be effective for projects that lack the existence of historical information for similar projects (Project Management Institute 2008). As part of the employees development program, members of the Project Management group in ORGUS delivered a series of training sessions on costing and estimation to support

this requirement, in addition to refresh sessions every quarter. The investigators reviewed 35 small projects for which the same personnel estimated the cost (17 prior to the training and 18 after the training). The results showed that over 66 % of the projects were delivered within 15 % cost tolerance in comparison to about 41 % with same cost tolerance.

“Limited time to release”

An agile team should work toward providing a bug-free release; the team should have ready-to-ship code at the end of any iteration (Watkins 2009). The item with the highest priority in the subsequent iteration should be to fix any remaining defects from the previous iteration (Stober and Hansmann 2010). Limited time to release could be a major factor that motivates the team to defer testing to the next release: the closer the release date, the greater the pressure on the team members (Humble and Farley 2011). Introducing automated testing, along with continuous delivery of smaller releases, saves time and effort, and reduces the need to work under pressure and stress (Humble and Farley 2011). Unfortunately, the automated testing was another example of the waste elimination strategies that were made during the *Deliberate Improvements* phase and left on the *Internal Implementation* phase.

3.4.8 Waste Classification (H): Defects

In IT context, a defect is defined as the unintended behaviour of a computer program or system. It is a common term that is used to describe an error, failure, or fault that produces an incorrect or unexpected result. Every system produced contains defects (Kim and Pries 2011). Defects are considered to be the main cause for rework and they also are an indication of product quality (Staats and Upton 2011; Besterfield 2009).

The elimination of defects from software depends upon the effectiveness and efficiency of the software V and V process. A very strong relationship exists between V and V and defects: the more frequent the V and V, the fewer the number of defects. Eliminating deferred V and V waste by introducing them in the early stages of a system development project will reduce the waste associated with defects. In addition, the cost of fixing the defects will be much lower if the defect is fixed at, or close to, the point where it was originally introduced. The waste and elimination strategies that we identified as part the “Defects” classification are:

“Inaccurate and obsolete specifications and conflict between supervisor directions and client’s requirements”

Software quality may be measured by the number of defects in customer acceptance testing (Staats and Upton 2011). The number of defects detected depends on system specifications. Ironically, some people working in the IT industry do not know the difference between user requirements and system specifications. Evidence for this exists in online programmer and developer forums: in one example, a discussion about differences between requirements and specifications in IT has seven answers and about 4,000 views, the discussion ended with no agreement on what the difference between requirements and specifications are (Programmers Forum 2012). This discussion raised an interesting and important question: if people within the IT industry cannot differentiate between requirements and specifications, how can we expect them to develop appropriate specifications?

In short, user requirements describe what the user needs—what the system or finished product must deliver (Project Management Institute 2008). User requirements are measured, or “validated”, against the user’s acceptance criteria (Besterfield 2009) and answer the question,

“Are we building the right product?” (Watkins and Mills 2011). System specifications, on the other hand, are the criteria that guide the actual system development work; in other words, they are the admissible or tolerance limits established by the project team members and supervisors (Project Management Institute 2008; Besterfield 2009). These criteria are used to determine the quality of the product under development by means of the verification process and answer the question, “Are we building the product right?” (Watkins and Mills 2011).

The directions from supervisors should be deriving and contributing to the specifications while the customers requirements should be determined in cooperation and consultation with the customer, in case of a conflict, the issue should be discussed with the stakeholder (supervisor and customer) to reach a consensus, if no agreement has been achieved, the issue should be escalated to the higher level.

For example, a customer requirement was to have the ability to maintain security groups within a specific system, however, the security specifications indicated that only system administrators were privileged to this functionality amongst other configuration privileges that should be retained within ORGUS. After negotiation, the customer accepted to pay the cost of developing a customized role to manage the security groups only and banned from the other configuration privileges.

“Excessive quantitative productivity standards for evaluation”

The employee performance appraisal system in ORGUS contributed to creating defects that required rework. The employee was being measured against such criteria as the number of tickets/problems resolved, the number of telephone calls answered, the number of lines of code written, the number of pieces of equipment/hardware repaired, it was obvious that every criterion asked for volume, thereby promoting a focus on quantity over quality. Changing the evaluation process to one that focuses on quality of employee performance must start by understanding the employees, their experience, and abilities (McGovern and Shelly 2008).

Senior management in ORGUS established a new approach for performance evaluations and introduced one-on-one (“O3”) meetings between the employee and the supervisor to build stronger relationships and remove barriers. Supervisors started to have a better understanding of their employees’ experience and abilities. Questions that are asked, as part of the performance appraisal process, have been restructured to focus on how well the work was done and how the results could be improved. The criterion for performance evaluation has changed to address such things as how cost can be reduced while increasing the quality of the service. As well, customer evaluations for the services that are provided were implemented to assess the employees’ qualitative performance.

“Unclear customer requirements, resulting in defective software”

The root causes for this waste, specifically, the lack of customer involvement resulting in inappropriate assumptions; this issue was addressed previously in Section 3.4.3 (C). This is just an example of how the elimination of root causes wastes (lack of customer involvement and inappropriate assumptions) lead to eliminating the subsequent waste (unclear customer requirement, resulting in defective software).

3.4.9 Waste Classification (I): Outdated Information / Obsolete Working Version

Traditional approaches to software development do not typically incorporate the continuous delivery philosophy. Developers are accustomed to having a long development cycle time; the problems associated with using outdated information or obsolete working versions, therefore,

only surface when the developers start to commit their changes and integration testing starts (Humble and Farley 2011). The waste and elimination strategies that we identified as part of the “Outdated information / Obsolete working versions” waste classification are:

“Developers do not integrate frequently enough”

Unexpected defects are usually discovered because of the recently committed work by other developers since the last retrieval of the working version or because of changes to the environmental settings amended to the latest available working version used by the developer. This requires rework on the changes made by the developer to fix the breakage to the trunk – the parent working version – this might be an infinite cycle if the developers do not change their mindset and start working in a continuous development environment. We recommended the use of a version control system and a continuous integration server for the software development team and we increased the organization’s awareness of the importance of using a continuous delivery philosophy. By reviewing the version control commit log messages, the developers showed a higher keenness towards committing their changes, which has reached 4 commits per day per developer, which in turn helped in reducing the defect rate from 14 bugs per thousand lines of code (KLOC) to 8 bugs per KLOC.

“Missing, inaccurate, or incomplete documentation for complex systems”

During the period of time that services were transitioning in ORGUS, we identified the deliverables that were necessary for the new team(s) to run the service efficiently. These included configuration documents, training, and service and system documentation. Unfortunately, based on a survey questionnaire with more than 50 participants, we found that about 70 % of the services and systems lacked adequate and up-to-date documentation. The survey included questions, about documentation structure, clarity, tone, consistency, terminologies and unambiguity, the questions included:

- 1) Does the documentation include table of contents or indexes?
- 2) Is the documentation structure simple and clear?
- 3) Does it include titles and subtitles?
- 4) Does it indicate where are the physical components located? (Datacenter location, physical server location, cabinet and rack number...Etc.)
- 5) Does it indicate how to get access? (How to get credentials, IP addresses, Etc.)
- 6) Are all links clear enough to tell where they will take the user?
- 7) Are the links functional? Or are they broken and not found?
- 8) Does the documentation provide clear, numbered step-by-step instructions?
- 9) Are technical terms used consistently?
- 10) Are sentences short and simple? Can non-native speakers understand them?

The Lean philosophy “Do it right the first time” (Hino 2002; Dennis 2007; Womack 2007) has a flip side of “Do it right, or do not do it at all”. Since one of the pre-transition requirements was to provide full and up-to-date documentation of the service under transition, the schedule, cost, and effort for the transitioning project were revised to include the development of the documentation to the project scope.

The lack of documentation raised another issue. What would happen if the current employee responsible for the IT service were to leave ORGUS? This was a main concern for those services that were more complex and that lacked proper documentation: in fact, we

identified some services that ORGUS will not be able to provide, if the employee who runs them leaves ORGUS. In order to mitigate this risk, the employees responsible for running these services were required to train at least two additional employees to provide these services. Other knowledge sharing strategies could also be applied.

“Distributing reports too early, with information that is quickly obsolete”

The new reporting structure and template were helpful in eliminating the waste associated with this category. As well, different types of weekly reports are now being populated automatically based on the stakeholders’ pre-identified requirements. The use of a shared collaborative document environment meant that reports no longer had to be sent as attachments by email. Stakeholders can now access up-to-date project reports at any time.

4 Performance Observations from the Lean Initiative

As part of the *Implement and Observe Improvements* phase to validate the results of the study, the investigators assessed performance of ORGUS using a Business Intelligent (BI) reporting and analysis tool integrated with the ticketing system. Furthermore, customers’ satisfaction survey was also considered to be the base for the improvements measurements.

4.1 Performance Improvements

For the *Request for Change (RFC)* type of services, this study included 1,684 requests for change between January 2012 and December 2012. The lead-time dropped from an average of 58 days, and 59 days for the first and second quarters, respectively, to an average of 33 and 23 days, for the third and fourth quarters, respectively. On average, ORGUS reduced lead-time by 60 % and achieved the targeted level performance of 30 days lead-time. Figure 7 illustrates the improvements achieved.

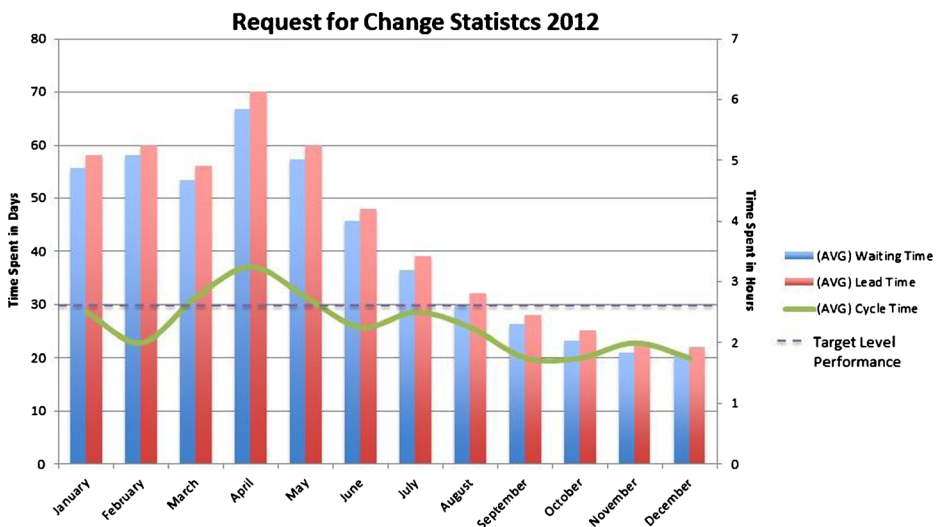


Fig. 7 Request for changes improvements

For the *Service Request (SR)* type of services, this study included 67,546 service requests between January 2012 and December 2012. The lead-time dropped from an average of 5.5 h and 6 h for the first and second quarters, respectively, to an average of 3.3 h and 2.6 h for the third and fourth quarters, respectively. On average, ORGUS reduced its lead-time by 56 % and achieved the targeted level performance of 3 h lead-time. Figure 8 illustrates the improvements achieved.

In addition, the customer surveys showed an increment in the level of satisfaction throughout the year. On average the customer satisfaction rate increased from an average of 79 % and 81 % for the first and second quarters, respectively, to an average of 91.7 % and 94.7 for the third and fourth quarters, respectively. Figure 8 below illustrates the improvements achieved

The number of participants in the survey was 7,654 participants. The survey consisted of three simple questions:

- 1) Are you satisfied with the service(s) offered?
- 2) Did ORGUS provide the service(s) in a timely manner?
- 3) In the future, how can ORGUS serve you better? (Comments, suggestions, thoughts?)

Surveys were sent only once at the end of the month in which the service(s) were provided. For instance, if customer ABC submitted 24 SRs during the month of July, assume that the 18 SRs were closed during the same month, while the remaining 6 SRs were closed in August, the customer would receive only one survey for the SRs that were closed in July, and only one survey for SRs that were closed in August.

4.2 Validity Threats

Care needs to be taken when interpreting Figs. 7 and 8, as clearly no causal link can be guaranteed to exist between the Lean initiative and the performance measures. Nor

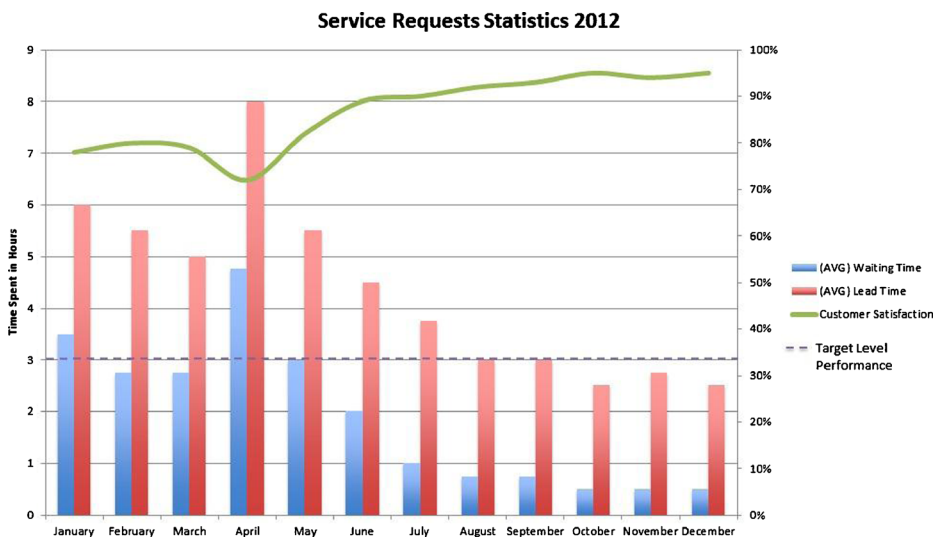


Fig. 8 Service requests improvements

can we discount the impact of confounding variables or local factors, such as the skills of the new hires or the improvements realized by other projects. These factors could have significant impacts on the presented results, which compromises a threat to the *Internal Validity*.

This study summarizes 18 months of work in one company, studying its culture, practices, policies and most importantly, its employees (*study participants*), the *External Validity* threat to our study is that our observations on this company will not generalize to other settings. In an effort to reduce the *Reliability* threat, the research study was undertaken by two researchers who made independent observations, as such the possibility of having erroneous results full of bias was reduced, however, the possibility of having biased interpretation of observations still a valid threat.

In an attempt to reduce the *Construct Validity* threat, field studies data collection techniques were used during the three phases of the *Action Research Cycle* to triangulate data in order to limit data misinterpretation and to validate the findings (Easterbrook et al. 2008; Dittrich et al. 2008). The investigators had written the observations, and asked 35 observed participants to read and validate them.

Direct techniques were also applied, which included semi-structured interviews, but mainly observational: *Think-aloud* and *participant observer* protocols (Singer et al. 2008); As per ORGUS' policies, all meetings and group activity sessions should be video recorded for future references, the researchers had been given access to review the video recording, which could be categorized under *Fly on the wall*, indirect source of data (Singer et al. 2008). The independent techniques that were employed to collect data included analysis of tool logs and documentation analysis (Singer et al. 2008). The application of these techniques to the data triangulation from multiple resources has added another dimension to strengthen the validity of the study.

5 Conclusions

Previous research has created IT-specific terms and phrases to represent the seven types of waste that have been identified in manufacturing environments. For example, the waste “Over-processing” in manufacturing is called “extra development steps” in an IT environment. Similarly, the manufacturing term “inventory” is simply translated into “requirements” (Schyff 2011). Unfortunately, the terms reflect a mapping of terms from manufacturing environment rather than being discovered from an IT environment. To understand what the wastes in the IT industry are, research is needed to analyze the activities within IT operations. A Lean initiative is not a “one size fits all” project.

Table 3 provides a summary of the proposed classification model with related examples identified in ORGUS, some examples are followed by notes to explain the rationale behind classifying the identified waste under a specific proposed classification.

Waste identification and elimination strategies may be done in an iterative fashion. “Thinking lean” when applying the Lean principles is the smartest and shortest path for a successful Lean journey. Making use of proven methodologies and techniques to eliminate waste is a Lean way to implement Lean; for instance, employing continuous and incremental improvements have proven to be efficient in reducing cost and increasing quality (Petersen and Wohlin 2009).

Table 3 Summary of the proposed classification model with examples

ID	Classification	Identified waste
1.1	Gold plating	Unnecessary tools, technologies, or methodologies
1.2		Producing and distributing reports that are not required, and providing more information than necessary
1.3		Unnecessary cosmetic functionality designed and developed into software
1.4		Striving to design perfect systems, rather than designing systems that simply are adequate
2.1	Over-specifications	Excessive analysis that does not support decision-making
2.2		Searching for information that is difficult to find
2.3		Overdesign of software or application
3.1	Lack of customer involvement and inappropriate assumptions	Misunderstanding of user requirements
3.2		Inflexible and lengthy approval process that encourages work-arounds.
3.3		Work performed earlier than needed because of the absence of prioritization and dependencies (result of assumptions)
3.4		Lost information, thus, developer starts to make assumptions
3.5		Limited access to the customer
3.6		Onsite visits to solve problems that could be solved remotely
4.1	Double handling / duplicate processes	Providing the same services by more than one team
4.2		Having redundant Data in the same or different systems
4.3		Multitasking, in which people move between incomplete projects because of the absence of project prioritization
4.4		Repetitive, unnecessary, and ineffective meetings
4.5		Troubleshooting to solve single problem instead of root cause
4.6	Centralized decision making	Focusing on team optimization not overall departmental performance
5.1		Unnecessary approval processes and unclear responsibility and authority
5.2		Complex reporting systems that add extra effort without adding value
5.3		Making decisions that lock in resources too early and produce inflexibility
5.4		Lengthy approval processes that discourage innovation and creativity
5.5		Lack of delegation to empower the employees resulting in technical decisions made by non-technical people
6.1	Waiting	Waiting for review and approval on action items
6.2		Waiting for information that is needed complete a task
6.3		Using shared resources; waiting to assign specialists to accomplish tasks
6.4		Developer waits for testing to finish before starting work on the next piece of code
6.5		Waiting for a physical signature
6.6		Waiting to identify the team for providing problem resolution
6.7		Clients waiting on the phone for their calls to be transferred to the appropriate team
7.1	Deferred verification and validation	Lack of system integration between legacy systems (Waterfall instead of Agile)
7.2		Excessive standards to follow that encourage avoiding testing
7.3		Not enough training on, or awareness of, the importance of testing

Table 3 (continued)

ID	Classification	Identified waste
7.4	Defects	Insufficient funds for testing based on budget underestimation
7.5		Limited time to release
8.1		Inaccurate and obsolete specifications and conflict between supervisor directions and client's requirements
8.2		Excessive quantitative productivity standards for evaluation
8.3		Unclear customer requirements, resulting in defective software (apart from customer involvement, requirements might be misunderstood or unclearly documented)
9.1	Outdated information / obsolete working version	Developers do not integrate frequently enough
9.2		Missing, inaccurate, or incomplete documentation for complex systems
9.3		Distributing reports too early, with information that is quickly obsolete

A successful Lean initiative needs senior management support and employee buy-in since a waste identification process will trigger the need to start new projects and these in turn will trigger significant changes at all levels in the organization. Senior management should be aware that to implement Lean successfully, a large investment maybe required to carry out different projects initiated throughout the Lean journey.

Table 4 provides a summary of the possible solutions that were developed during the *Deliberate Improvements* phase along with the decision that was taken either to move to *Implement and Observe Improvements* phase or just to leave the recommendations in the *Deliberate Improvements* phase for future considerations based upon the resources availability in ORGUS:

ORGUS has had several tools and systems in place that where considered when developing the improvement strategies, which helped in reducing the cost to implement these strategies and led to maximizing senior management support. The most challenging factor we faced during the project was, the changes to the *organizational defensive routines* and the changes to the employees' *theory-in-use*. The change process was not easy to implement, however, it was achievable by implementing *double-loop learning*. Showing quick wins, in order to develop employees' buy-in and management support, was a critical factor to success. We began with the small changes that would have eliminated obvious wastefulness processes to achieve large performance improvements.

Lean practitioners need to take into consideration the limitations and challenges of the organization when developing the strategies to identify and eliminate wastes. The investigators worked collaboratively with *research participants* to design and develop improvements strategies and recommendations to identify and eliminate wastes based upon the time and resources available in ORGUS. The model developed during this project could be used as starting point for IT organizations that would like to tackle a lean initiative, however, these organizations should analyze their own process to discover the wastefulness processes within their core IT operations.

Table 4 Summary of the proposed techniques to eliminate wastes

ID	Possible solution	Phase
1.1	Using SIPOC-Flowchart and MoSCoW Prioritization	I
1.2	Creating a report template that contains the information that the recipients deem to be necessary	I
1.3	Introducing Kanban board and Stand-up meetings	I
1.4	Implementing <i>Double-loop learning</i> and Stand-up meetings	I
2.1	Implementing <i>Double-loop learning</i> , Stand-up meetings, and reporting	I
2.2	Stating all assumptions and document them as potential risks	I
2.3	Establishing customer involvement/feedback model	D
3.1	Hiring competent business analysts and customer feedback	I
3.2	Changing the <i>organizational defensive routines</i> (Argyris 2004)	I
3.3	Establishing prioritization process i.e. MoSCoW prioritization	I
3.4	Using backup, version control, and document management systems – in ORGUS, tools were existed but not utilized!	I
3.5	Establishing customer involvement/feedback model	D
3.6	Using remotely access software tools	I
4.1	Establishing SPOC and SQM	I
4.2	Establishing central data repository or document management system - in ORGUS, tools were existed but not utilized!	I
4.3	Establishing prioritization process i.e. MoSCoW prioritization	I
4.4	Developing and implementing meeting rules	I
4.5	Implementing root-cause analysis	I
4.6	Changing the <i>organizational defensive routines</i> (Argyris 2004)	I
5.1	Establishing RACI matrix and Changing the <i>organizational defensive routines</i> (Argyris 2004)	I
5.2	Using automated reporting tool – in ORGUS MS Excel is used	I
5.3	Implementing collaborative-decision making environment	D
5.4	Changing the <i>organizational defensive routines</i> (Argyris 2004) to empower employees – in ORGUS, ITIL framework helped in reducing/eliminating some unnecessary approval steps	I
5.5	Define/Revise processes with domain experts	I
6.1	Changing the <i>organizational defensive routines</i> (Argyris 2004) to empower employees – in ORGUS, ITIL framework helped in reducing/eliminating some unnecessary approval steps	I
6.2	Establishing document management system to create knowledge base- in ORGUS, tools were existed but not utilized! In addition, ITIL helped in creating knowledge bases as its one of the required systems in ITIL	I
6.3	Implementing collaborative-decision making environment	D
6.4	Introducing Kanban board and Stand-up meetings	I
6.5	Using Digital Signature	D
6.6	Establishing SQM	I
6.7	Establishing automated call messaging system	D
7.1	Employing the Continuous Integration technique	D
7.2	Establishing “self-accessed policies”	D
7.3	Changing the <i>organizational defensive routines</i> (Argyris 2004) – in ORGUS, ITIL framework helped in increasing the awareness testing viability as well as ITIL enforced preparing test plans prior final approval of <i>Major Changes</i>	I

Table 4 (continued)

ID	Possible solution	Phase
7.4	Changing the <i>organizational defensive routines</i> (Argyris 2004) - PM group in ORGUS delivered a training program on costing and estimation	I
7.5	Employing the automated testing and continuous integration technique	D
8.1	Changing the <i>organizational defensive routines</i> (Argyris 2004) - Establishing specifications that meet the quality level required by the customer and make sure that the specifications do not conflict with requirements. Empowering employees to define specifications along with establishing robust customer involvement model realize the changes.	D
8.2	Changing the <i>organizational defensive routines</i> (Argyris 2004) – ORGUS changed employee's evaluation process to one that focuses on quality not quantity, customer feedback is used now as part of the employee's annual evaluation	I
8.3	Establishing customer involvement/feedback model	D
9.1	Establishing of a version control system and a continuous integration server for the software development team	
9.2	Employing the Continuous Integration technique	D
9.3	Using automated reporting tool and document management system	I

D Deliberate Improvement

I Implement and Observe Improvements

References

- Addy R (2007) Effective IT service management: to ITIL and beyond! Springer-Verlag Berlin Heidelberg, New York
- Ammerman M (1998) The root cause analysis handbook: a simplified approach to identifying, correcting, and reporting workplace errors. Productivity, Inc., Portland
- Argyris C (2004) Reasons And Rationalizations: The Limits to Organizational Knowledge. Oxford University Press
- Bellingham R (2004) HR optimization: from personnel administration to human and organizational capital development. HRD Press, Inc., Massachusetts
- Besterfield DH (2009) Quality control. Pearson Prentice Hall, New Jersey
- Bhagat RS, Steers RM (2009) Cambridge handbook of culture, organizations, and work. Cambridge University Press, New York
- Boeg J (2011) Priming kanban: a 10 step guide to optimizing flow in your software delivery system. Chronografisk, Denmark
- DeMarco T (2011) All late projects are the same. IEEE Softw 28(6):103–104
- Dennis P (2007) Lean production simplified. Productivity Press, New York
- de-Ste-Croix A, & Easton A (2008) The Product Owner Team. Agile '08 Conference (pp. 274–279). IEEE
- Dittrich Y, Rönkkö K, Eriksson J, Hansson C, Lindeberg O (2008) Cooperative method development. Empir Softw Eng 13(3):231–260
- Easterbrook S, Singer J, Storey M-A & Damian D (2008) Selecting Empirical Methods for Software Engineering Research. In Guide to Advanced Empirical Software Engineering (pp. 285–311). Springer London
- Goldratt EM, Cox J (1992) The goal: a process of ongoing improvement. North River Press, Massachusetts
- Harvard Business Review (2011) Management tips. Harvard Business Press, Massachusetts
- Hino S (2002) Inside the mind of Toyota. Productivity Press, New York
- Humble J, Farley D (2011) Continuous delivery. Addison-Wesley, New Jersey
- International Institute of Business Analysis (2009) A Guide to the Business Analysis Body of Knowledge (BABOK)
- Jaisingh S (2012) Lean manufacturing [class handout]. department of mechanical engineering, University of Alberta. Edmonton, Canada
- Kim H, Pries J (2011) Testing complex and embedded systems. CRC Press, New York
- Liker JK, Hoseus M (2008) Toyota culture: the heart and soul of the Toyota Way. McGraw-Hill, New York
- Magee D (2008) How Toyota became #1. Penguin Group, New York

- Martin A, Biddle R & Noble J (2009) XP Customer Practices: A Grounded Theory. Agile Conference, AGILE '09. (pp. 33–40). IEEE
- McConnell S (1996) Rapid development: taming wild software schedules. Microsoft Press, Washington
- McGovern J, Shelly S (2008) The happy employee: 101 ways for managers to attract, retain, & inspire the best and brightest. Adams Business, Massachusetts
- Middleton P (2012) Lean software management: BBC worldwide case study. IEEE Trans Eng Manag 59(1):20–32
- Morgan JM & Liker JK (2006) The Toyota Product Development System: Integrating People, Process, and Technology. Productivity Press
- Murugaiah U, Benjamin SJ, Marathamuthu MS, Muthaiyah S (2010) Scrap loss reduction using the 5-whys analysis. Int J Qual Reliability Manag 27(5):27–540
- Nottonson K (2011) Yahoo! sports: sprint 100 & beyond. agile conference (pp. 252–255). IEEE
- Ohno T (1988) Toyota production system: beyond large-scale production. Productivity Press
- Packlick J (2007) The agile maturity map a goal oriented approach to agile improvement. agile conference 2006 (pp. 266–271). IEEE
- Petersen K, Wohlin C (2009) A comparison of issues and advantages in agile and incremental development between state of the art and an industrial case. J Syst Softw 82(9):1479–1490
- Petersen K, Wohlin C (2010) Software process improvement through the lean measurement (SPI-LEAM) method. J Syst Softw 83(7):1275–1287
- Poppendieck M & Poppendieck T (2003) Lean software development: an agile toolkit. Addison Wesley
- Poppendieck, M & Poppendieck T (2006) Implementing lean software development: from concept to cash. Addison Wesley
- Programmers Forum (2012) What is the difference between requirements and specifications? Retrieved October 20, 2012, from <http://programmers.stackexchange.com/questions/121289/what-is-the-difference-between-requirements-and-specifications>
- Project Management Institute (2008) A guide to the project management body of knowledge (PMBOK guide). USA, Philadelphia
- Przekop P (2006) Six sigma for business excellence: a Manager's guide to supervising six sigma projects and teams. McGraw-Hill, New York
- Rand C & Eckfeldt B (2004) Aligning strategic planning with agile development: extending agile thinking to business improvement. agile development conference, (pp. 78–82)
- Runeson P, Höst M (2009) Guidelines for conducting and reporting case study research in software engineering. Empir Softw Eng 14(2):131–164
- Schyff A (2011) Lean software development
- Schwaber K, Sutherland J (2012) Software in 30 days: how agile managers beat the odds, delight their customers, and leave competitors in the dust. Wiley, NJ
- Shah R, Ward PT (2007) Defining and developing measures of lean production. J Oper Manag 25(4):785–805
- Shelton C (2010) Business process reengineering with commercial off-the-shelf software. department of defense, department of the air force Washington DC. Defense AT&L, Washington
- Singer J, Sim SE, Lethbridge TC (2008) Software engineering data collection for field studie. In: Singer J, Sim SE, Lethbridge TC, Shull F, Singer J, Sjøberg DI (eds) Guide to advanced empirical software engineering. Springer, London, pp 9–34
- Spearman WJ (2004) To pull or not to pull: what is the question? Manuf Serv Oper Manag 6(2):133–148
- Staats BR, & Upton DM (2011) Lean knowledge work. Harvard business review
- Staats BR, Brunner DJ, Upton DM (2011b) Lean principles, learning, and knowledge work: evidence from a software services provider. J Oper Manag 29(5):376–390
- Stober T, Hansmann U (2010) Agile software development: best practices for large software development projects. Springer-Verlag Berlin Heidelberg, New York
- Tapping DT (2002) Value stream management: eight steps to planning, mapping, and sustaining lean improvements. Productivity Press, New York
- Walsh R (2008) Time management: proven techniques for making every minute count. Adams Media, MA
- Wang X, Conboy K, Cawley O (2012) “Leagile” software development: an experience report analysis of the application of lean approaches in agile software development. J Syst Softw 85(6):1287–1299
- Waterhouse P (2008) Improving IT economics: Thinking ‘Lean’. CA White Paper
- Watkins J (2009) Agile testing: How to succeed in an extreme testing environment. Cambridge University Press, New York
- Watkins J, Mills S (2011) Testing IT: an Off-the-shelf software testing process, 2nd edn. Cambridge University Press, New York
- Womack JP (2007) The machine that changed the world. Free Press, New York
- Xingjian Y (1992) A database design method for finite element analysis. Comput & Struct 44(4):911–914
- Yap M (2006) Value-based extreme programming. Agile conference (pp. 175–184). IEEE
- Younker J (2008) Foundations of agile python development. Apress



Osama Al-Baik is currently a doctoral candidate at University of Alberta in Canada, his research interests have been in Software and Systems Engineering, Lean Software Development, Software Process Improvement (SPI), and Software Project Management. He has been working for more than 10 years in software development and project management for multinational companies, where he has held senior positions including, Managing Director, General Manager and Director of Project Management Office. Osama has a Bachelor's degree in Computer Information Systems from Amman University, Jordan and a Master's degree in Software Engineering/Project Management from DePaul University, USA. He is PMP certified from PMI and a certified project manager (CPM) from IAPPM.



James Miller received the BSc and PhD degrees in computer science from the University of Strathclyde, Scotland. During this period, he worked on the ESPRIT project GENEDIS on the production of a real-time stereovision system. Subsequently, he worked at the United Kingdom's National Electronic Research Initiative on Pattern Recognition as a principal scientist, before returning to the University of Strathclyde to accept a lectureship and, subsequently, a senior lectureship in computer science. Initially, during this period his research interests were in computer vision, and he was a co-investigator on the ESPRIT 2 project VIDIMUS. Since 1993, his research interests have been in web, software, and systems engineering. In 2000, he joined the Department of Electrical and Computer Engineering at the University of Alberta, Canada, as a full professor and in 2003 he became an adjunct professor in the Department of Electrical and Computer Engineering at the University of Calgary. He is the principal investigator in a number of research projects that investigate software verification, validation and evaluation issues across various domains, including embedded, web-based, and ubiquitous environments. He has published more than 100 refereed journal and conference papers on web, software, and systems engineering (see www.steam.ualberta.ca for details on recent directions). He recently served as the 2011 organizing chair for the IEEE International Symposium on Empirical Software Engineering and Measurement, and he sits on the editorial board of the Journal of Empirical Software Engineering. He is a member of the IEEE.